

Unit 1:

Introduction To Finite Automata

- Central concepts of automata theory
- Deterministic finite automata
- Non-Deterministic finite automata

Finite Non-Empty Set of Symbols

Basic Notation and terminologies used in formal language and automata theory.

1] Alphabet: An alphabet is defined as a finite non empty set of symbols. This symbol Σ denotes the set of alphabets of a language.

Ex: *The alphabet of C language

$$\Sigma = \{A, B, C, \dots, a, b, \dots, \epsilon, 0, 1, \dots, 9, \#, (,), <, >\}$$

*The alphabet of machine language

$$\Sigma = \{0, 1\}$$

2] String: The sequence of symbols obtained from the alphabets of a language is called a string. It is a finite sequence of symbol from the alphabet Σ .

An empty string is denoted by the symbol ϵ or λ where $\epsilon \notin \Sigma$ i.e; ϵ is not part of Σ .

Ex: Let $\Sigma = \{0, 1\}$ is set of alphabets.

The various strings that can be obtained from Σ are

$$\{0, 1, 00, 01, 10, 11, 010101, 1010, \dots\}$$

3] Concatenation: Concatenation of 2 strings u and v is the string obtained by written writing the letters of the string u followed by the letters of string v [i.e; appending the symbols of v the right of u] consider two strings.

$$u = a_1 a_2 a_3 \dots a_n$$

$$v = b_1 b_2 b_3 \dots b_m$$

So concatenation of u and v and v and u is given by $uv = a_1 a_2 a_3 \dots a_n b_1 b_2 b_3 \dots b_m$
 $vu = b_1 b_2 b_3 \dots b_m a_1 a_2 a_3 \dots a_n$

Concatenation is not commutative

ex: let $u = \text{computer}$
 $v = \text{science}$

$$uv = \text{ComputerScience}$$

$$uv \neq vu$$

4] Substring: Let w is a string obtained from the symbols in Σ the string w will become string to 3 strings x, y and z such that $w = xyz$ then x is a substring, y is a substring and z is a substring of string w .

5] Prefix: A prefix is a string of any number of leading symbols.

ex: $w = \text{Suma}$

prefixes are $\epsilon, s, su, sum, suma$.

6] Suffix: Suffix is a string of any number of trailing symbols.

ex: $w = \text{Suma}$

suffixes are $\epsilon, a, ma, uma, suma$

7) Reverse of a string: It is obtained by writing the symbols in reverse order.

If $u = a_1 a_2 a_3 \dots a_n$, then the reverse of u is given by $u^R = a_n a_{n-1} a_{n-2} \dots a_3 a_2 a_1$.

If u is an empty string denoted by ϵ and reverse of u is given by ϵ^R .

If u has only one symbol then reverse of u is given by $a^R = a$.

8) Length of string: The length of the string u is the number of symbols in u and is denoted by $|u|$.

If $u = a_1 a_2 a_3 \dots a_n$, $|u| = n$.

$u = \text{STJIT}$, $|u| = 5$.

The length of an empty string ϵ is 0 and is denoted by $|\epsilon| = 0$.

9) Power of an alphabet: Power of an alphabet denoted by Σ to the power of i is the set of words of length i .

If $\Sigma^0 = \{\epsilon\}$ denotes set of words of length 0.

$\Sigma^1 = \{0, 1\}$ denotes set of words of length 1.

$\Sigma^2 = \{00, 01, 10, 11\}$ denotes set of words of length 2.

$\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$.

Ex: $\Sigma = \{a, b\}$

$\Sigma^0 = \{\epsilon\}$

$\Sigma^1 = \{a, b\}$

$\Sigma^2 = \{aa, ab, ba, bb\}$.

$\Sigma^3 = \{aaa, aab, aba, baa, abb, bba, bbb\}$.

Two Variation Of Power of an alphabet:

① Kleene closure

② Kleene plus

Kleene Closure:

It is defined as Σ^*

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$$

which is the set of words of only length each string is made up of symbols only from Σ

ex: $\Sigma = \{a, bc\}$

$$\Sigma^0 = \{\epsilon\}$$

$$\Sigma^1 = \{a, bc\}$$

$$\Sigma^2 = \{aa, abc, bca, bc bc\}$$

$$\Sigma^* = \{\epsilon, a, bc, aa, abc, bca, bc bc, \dots\}$$

which is the set of strings of a's and b's of any length.

Kleene plus: It is a variation of Kleene star operator

The Kleene plus is denoted by Σ^+ is defined as follows

$$\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \cup \dots$$

which is the set of words of any length except the null string i.e. $\epsilon \in (\Sigma)$ is not part of Σ^+ and hence

$$\epsilon \notin \Sigma^+$$

ex: let $\Sigma = \{0, 1\}$ the Σ^+ is shown below

$$\Sigma^+ = \{0, 1, 00, 01, 10, 11, 000, 001, \dots\}$$

Language: It is defined as a set of strings obtained from Σ^* where Σ is set of alphabets of a particular language.

A language is subset of Σ^* . It is denoted by $L \subseteq \Sigma^*$

ex: ① Language which contains strings equal number of 0's and 1's

$$L = \{\epsilon, 01, 10, 0011, 1100, 0101, \dots\}$$

② Language of strings consisting of 'n' number of 0's followed by 'n' number of 1's

$$\Sigma = \{0, 1\}$$

$$L = \{01, 0011, 000111, \dots\}$$

③ Language containing an empty string ϵ is denoted by $L = \{\epsilon\}$

* Empty language is denoted by $\emptyset$

④ Language of strings consisting of even number of 1's

$$L = \{0011, 1100, 11110000, 11001100, 11, \dots\}$$

⑤ Language of strings that end in 'aa' for an alphabet $\Sigma = \{a, b\}$

$$L = \{baa, aa, babaa, \dots\}$$

Sentence: A string that belongs to a language is called word or sentence of the language.

ex: Language of strings consisting of equal number of 0's and 1's can be represented as

$$\Sigma = \{\epsilon, 01, 10, 0011, 1010, 0101, \dots\}$$

Each word separated by comma is a sentence.

Types Of Languages:

Type 0 language (phrase structured)

Type 1 language (context sensitive)

Type 2 language (context free)

Type 3 language (Regular language)

→ DFA
NFA
E-DFA
Regular exp

Finite Automata: [Automaton] It is the mathematical model which is used to study the abstract machines or abstract computing device with the inputs chosen from an alphabet Σ .

On reading the string the machine may accept the string or may reject the string.

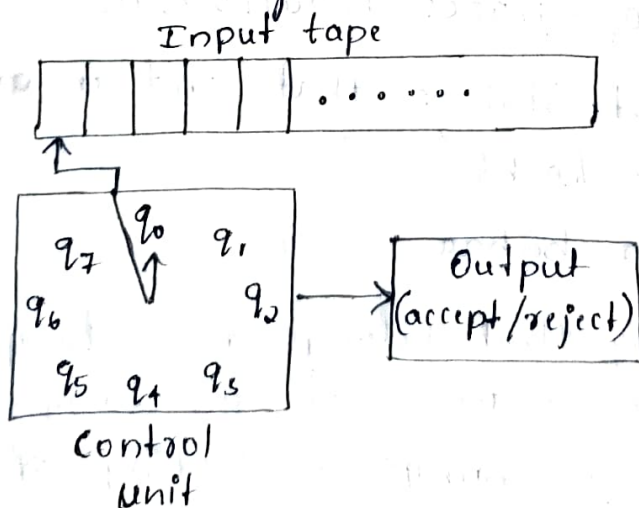
FA has three components as shown below:

→ Input tape

→ Control Unit

→ Output.

The block diagram of FA is shown below:



Input tape:

The input tape is divided into cells each of which can hold one symbol.

The string to be processed is stored in these cells

Control Unit:

Machine has some states one of which is the start state. Designated as q_0 and atleast one final state.

It has some finite states designated by $q_1, q_2, q_3, \dots$

Based on the current input symbol the state of the machine can change.

Output:

Output may be accept or reject when end of input is encountered the control unit may be in accept/reject state.

Working Of FA:

- Machine is assumed to be in start state q_0
- The input pointer points to the first cell of a tape pointing to the string to be processed.
- After scanning current input symbol the machine can enter into any of the state $q_0, q_1, q_2, \dots$

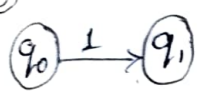
And the input pointer automatically points to the next character by moving one cell towards right.

- When the end of the string is encountered the string is accepted if and only if automata will be in one of the final states otherwise the string is rejected.

Notations Used during design of FA:

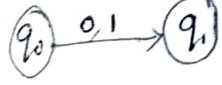
①  circles are used to represent the state of machine.

②  circle with an arrow which is not original from any node represents the start state of the machine.

③  An arrow with label 1 goes from state q_0 to state q_1 . This indicates there is a transition from state q_0 on input symbol 1 to state q_1 . $\therefore \delta(q_0, 1) = q_1$

④  An arrow with label 0 starts and ends in q_0 , this indicates the machine in state q_0 on reading 0 it remains in same state q_0 . $\delta(q_0, 0) = q_0$

⑤  represents final state.

⑥  An arrow with label 0, 1 goes from state q_0 to q_1 , this indicates the machine in state q_0 on reading 0 or 1 ends into state q_1 .
 $\delta(q_0, 0) = q_1$
 $\delta(q_0, 1) = q_1$

Types of FA :

- ① DFA [Deterministic FA]
- ② NFA [Non Deterministic FA]
- ③ ϵ -NFA [NFA with ϵ moves FA]

Deterministic Finite Automaton [DFA]



- The circles are called states or nodes or vertices. Each state is identified by a name q_0, q_1, q_2 .
Start state: q_0 with an arrow labelled start is considered as start state.
Final state: q_2 with two circles represent a final state.

- The above DFA has 3 states q_0, q_1 , and q_2 represented as $Q = \{q_0, q_1, q_2\}$

- DFA can have one or more final states. If there is no final state in DFA that means DFA accepts empty language.

- Input Alphabets: Each edge is labelled with 0 or 1 and represents the input alphabets which can be denoted as $\Sigma = \{0, 1\}$

- Transitions: Transition is change of state after consuming an input symbol. If there is a change of state from q_i to q_j on an i/p symbol a then we can write $\delta(q_i, a) = q_j$

$$\delta(q_0, 0) = q_0$$

$$\delta(q_0, 1) = q_1$$

$$\delta(q_1, 0) = q_1$$

$$\delta(q_1, 1) = q_2$$

$$\delta(q_2, 0) = q_2$$

$$\delta(q_2, 1) = q_2$$

In general $\delta(\Theta \times \Sigma) \rightarrow \Theta$ which is cross product of $\Theta = \{q_0, q_1, q_2\}$ and $\Sigma = \{0, 1\}$

Transition Function:

It is defined as $\delta(\Theta \times \Sigma) \rightarrow \Theta$ which is read as δ is transition function which maps $\Theta \times \Sigma$ to Θ

Ex: The change of state Θ from state q on input symbol 'a' to state 'p' is denoted by $\delta(q, a) = p$

$\delta \rightarrow$ transition function

$q \rightarrow$ first parameter representing current state of M

$a \rightarrow$ 2nd parameter representing current i/p symbol

$p \rightarrow$ represents next state of machine which is returned by the δ

$q_0 \rightarrow$ start state

$q_f \rightarrow$ final state

Definition: DFA is a 5-tuple or quintuple indicating 5 components $M = (\Theta, \Sigma, \delta, q_0, F)$ M is the name of the machine.

$\Theta \rightarrow$ non empty finite set of states

$\Sigma \rightarrow$ non empty finite set of input alphabets

$\delta \rightarrow$ Transition function which is mapping from

$\Theta \times \Sigma$ to Θ based on the current input sym

-bol the machine enters into another state.

$q_0 \rightarrow q_0 \in \Theta$ is the start state.

$F \rightarrow F \subseteq \Theta$ is a set of accepting or final states

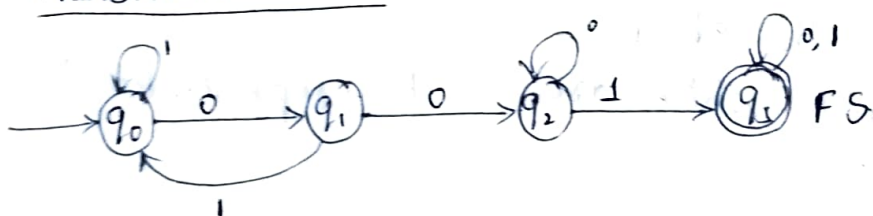
DFA :

D [Deterministic]: There is exactly one transition for every input symbol from the state so it is possible to determine exactly to which state the machine enters into after consuming the input symbol. So the machine is deterministic.

F [Finite]: Has finite number of states and transitions and edges so it is deterministic & finite.

A [Automaton]: Machine which may accept or reject the string so it is deterministic finite Automaton.

Transition Function:



$$\delta(q_0, 1) = q_0$$

$$\delta(q_0, 0) = q_1$$

$$\delta(q_1, 1) = q_0$$

$$\delta(q_1, 0) = q_2$$

$$\delta(q_2, 0) = q_2$$

$$\delta(q_2, 1) = q_1$$

$$\delta(q_3, 0) = q_3$$

$$\delta(q_3, 1) = q_3$$

Transition Diagram:

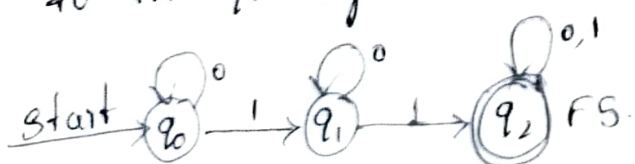
For DFA $M = (Q \in S \rightarrow F)$ is defined as graph with Q , arrow and arches with table, two circles etc.

- Each state of Q corresponds to one node or vertex represented using circle or two circles
- Alphabets
- The transition from one state to another state is indicated by the directed edges.
- Let $\delta(q_i, a) = q_j$ indicates that there is a directed edge from q_i to q_j at the edge is label a .
- The start state has an arrow not originating from any node and entering to the state there is labeled with start.
- The final state which are in F represented by double circle.

Transition Table:

Transition Table for the DFA $M = (Q \in S \rightarrow F)$ is defined as a tabular representation of a transition function such as δ which takes two arguments that are written

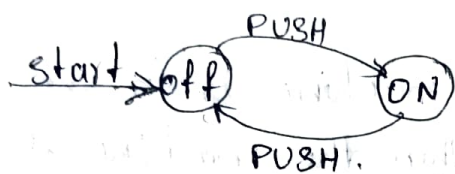
The rows of the table correspond to the state of DFA of time from Q columns correspond to the i/p symbol or ...



Transition table for:

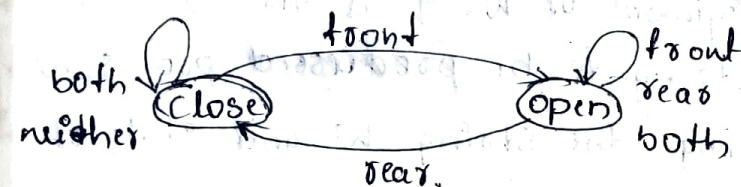
δ	0	1
q_0	q_0	q_1
q_1	q_1	q_1
q_2	q_2	q_2

* Transition of Switch condition



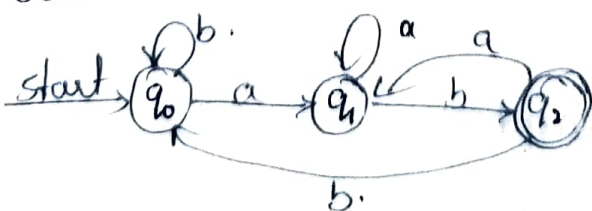
δ	PUSH
off	ON
ON	OFF

* Transition for automatic load:



How DFA transition:

That can be no transition from a state as shown below



From state q_0 or 1/p symbol 1 modline enters in to state q_2 from state q_2 , the transition is define ON any w of 1/p transition from state q_2

Extended Transition Of DFA input String:

The extended transition function δ^* describes what happens to a state of machine when the input is a string (sequence of symbols)

let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA. The extended transition function $\delta^*: Q \times \Sigma^* \rightarrow Q$.

Basic Step: $\delta^*(q, \epsilon) = q$

This indicates that if the machine is in the state q and we know ^{no} input then the machine is string in state q .

Induction: let $w = xa$ where a is the last symbol of w and x is remaining symbol of w . q is the current for p state and w is the string to be processed for p .

And after consuming the string w and state of p the machine is p .
Ex: $w = 1101$ $x = 110$
 $a = 1$

$$\delta^*(q, w) = \delta^*(q, xa) = \delta(\delta^*(q, x), a)$$

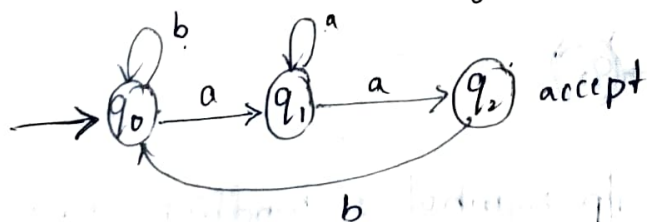
change of state from state q on input string w to state p is denoted by $\delta^*(q, w) = p$. | $\hat{\delta}(q, x) = p$

Here $\delta^* \Rightarrow$ extended transition function

$q \Rightarrow$ current state of the machine $\hat{\delta}(q, w) =$

$w \Rightarrow$ current i/p string to be processed $\hat{\delta}(q, w)$

$p \Rightarrow$ next state of the machine which is written by δ .

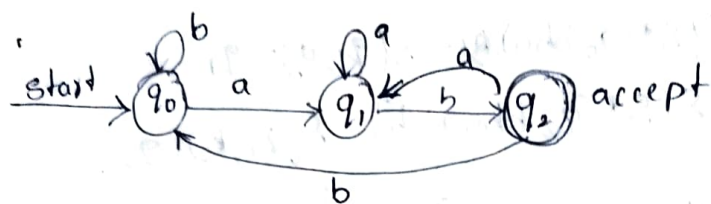


various properties of extended transition function are:

i) $\delta^*(q, \epsilon) = q$

ii) $\delta^*(q, w) = \delta^*(q, xa) = \delta(\delta^*(q, x), a)$ for $w = xa$

iii) $\delta^*(q, w) = \delta^*(q, ax) = \delta(\delta^*(q, a), x)$ for $w = ax$



i) abaab:

for prefix ϵ $\delta^*(q_0, \epsilon) = q_0$

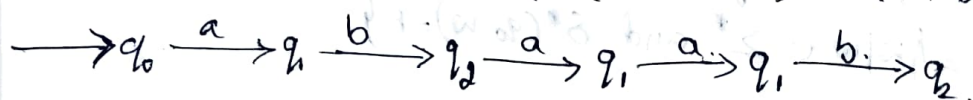
for prefix a $\delta^*(q_0, a) = \delta(\delta^*(q_0, \epsilon), a) = \delta(q_0, a) = q_1$

for prefix ab $\delta^*(q_0, ab) = \delta(\delta^*(q_0, a), b) = \delta(q_1, b) = q_2$

for prefix aba $\delta^*(q_0, aba) = \delta(\delta^*(q_0, ab), a) = \delta(q_2, a) = q_1$

$abaa$ $\delta^*(q_0, abaa) = \delta(\delta^*(q_0, aba), a) = \delta(q_1, a) = q_1$

$abaab$ $\delta^*(q_0, abaab) = \delta(\delta^*(q_0, abaa), b) = \delta(q_1, b) = q_2$



ii) abbab:

for prefix ϵ $\delta^*(q_0, \epsilon) = q_0$

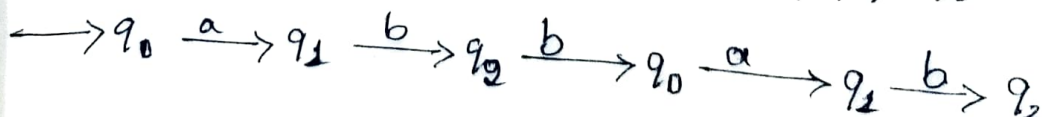
a $\delta^*(q_0, a) = \delta(\delta^*(q_0, \epsilon), a) = \delta(q_0, a) = q_1$

ab $\delta^*(q_0, ab) = \delta(\delta^*(q_0, a), b) = \delta(q_1, b) = q_2$

abb $\delta^*(q_0, abb) = \delta(\delta^*(q_0, ab), b) = \delta(q_2, b) = q_0$

$abba$ $\delta^*(q_0, abba) = \delta(\delta^*(q_0, abb), a) = \delta(q_0, a) = q_1$

$abbab$ $\delta^*(q_0, abbab) = \delta(\delta^*(q_0, abba), b) = \delta(q_1, b) = q_2$



DFA Design Techniques:

① Pattern Recognition Problems:

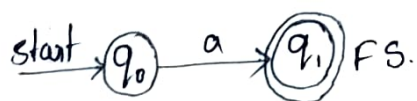
- General steps to be followed by designing DFA for pattern recognition problems
- ① Identify the minimum string
 - ② Identify the alphabets
 - ③ Construct skeleton DFA
 - ④ Identify other transitions not defined in step ③
 - ⑤ Construct DFA using transitions in step ③ & step ④

Ex ①: Draw a DFA to accept strings of a's have the atleast one a

Ans:

$$L = \{ \underline{a}, aa, aaa, aaaa, aaaaa, \dots \}$$

- ① The MS is a
- ② The alphabet $\Sigma = \{a\}$
- ③ Skeleton DFA is



$$\delta(q_0, a) = q_1$$

→ ④ $\delta(q_1, a) = ?$

so $\delta(q_1, a) = aa = \text{accepted} = q_1$

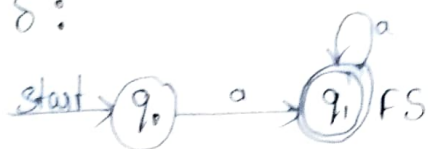
→ ⑤ $M = (Q, \Sigma, \delta, q_0, F)$

$$Q = \{q_0, q_1\}$$

$$\Sigma = \{a\}$$

Similarly for
atleast one 'b'
 $L = \{b, bb, bbb, bbbb, \dots\}$

δ :



$q_0 \Rightarrow$ start state

$F = \{q_1\}$

Transition table:

δ	a
$\rightarrow q_0$	q_1
$*q_1$	q_1

$$L = \{w : na(w) \geq 1, w \in \{a,b\}^*\}$$

ex 2: Draw a DFA to accept strings of a's and b's having at least one a

soln: $L = \{a, ab, ba, aab, bba, \dots\}$

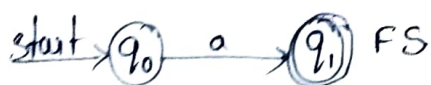
$bba, baa, bbaa, \dots$
 $aaa, \dots$

→ ① MS is a

→ ② $\Sigma = \{a, b\}$

$\{b, ba, bba, aab, bbb, \dots\}$

→ ③ skeleton DFA is:



$\delta(q_0, a) = q_1$

$$L = \{w : na(w) \geq 1, w \in \{a,b\}^*\}$$

→ ④ $\delta(q_0, b) = ?$, $\delta(q_1, a) = ?$, $\delta(q_1, b) = ?$

i) $\delta(q_0, b) = ba = \text{accepted} = q_0$

ii) $\delta(q_1, a) = aa = \text{accepted} = q_1$

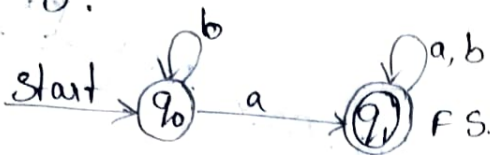
iii) $\delta(q_1, b) = ab = \text{accepted} = q_1$

→ ⑤ $M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1\}$

$\Sigma = \{a, b\}$

δ :

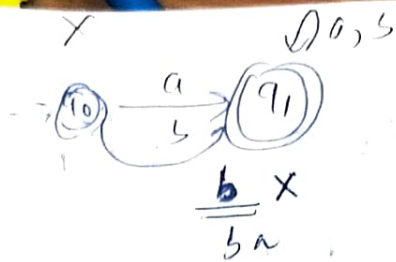


$q_0 \Rightarrow$ start state

$F = \{q_1\}$

Transition table:

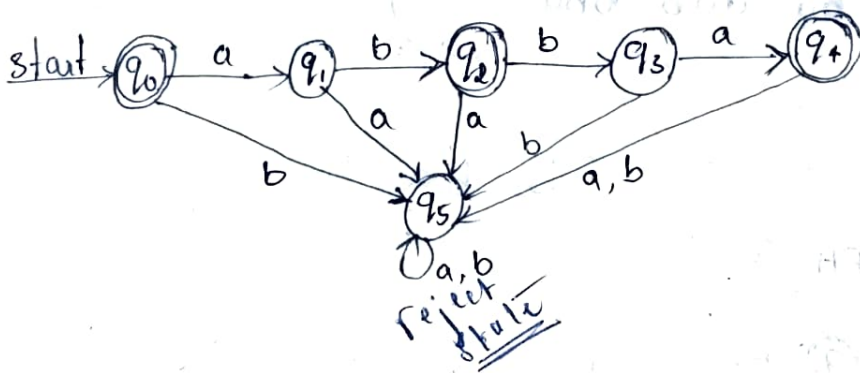
δ	a	b
$\rightarrow q_0$	q_1	q_0
$* q_1$	q_1	q_1



$L = \{w : n_a(w) \geq 1, w \in (a+b)^*\}$

Ex 3 Draw a DFA for $L = \{\lambda, ab, abba\}$.

Soln:



Ex 4: Draw DFA to accept strings a's and b's having exactly one a.

Soln: $L = \{a, ab, ba, abb, bab, \dots\}$

→ ① MS is a

→ ② $\Sigma = \{a, b\}$

→ ③ The skeleton DFA is



$\delta(q_0, a) = q_1$

→ ④ $\delta(q_0, b) = ?$, $\delta(q_1, a) = ?$, $\delta(q_1, b) = ?$

i) $\delta(q_0, b) = ba = \text{accepted} = q_0$

ii) $\delta(q_1, a) = aa = \text{rejected} = \emptyset$

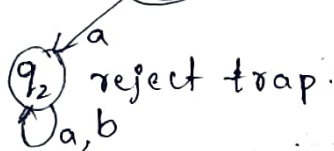
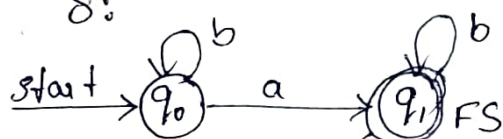
iii) $\delta(q_1, b) = ab = \text{accepted} = q_1$

→ ⑤ $M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1\}$

$\Sigma = \{a, b\}$

δ :



$q_0 \Rightarrow \text{start state}$

$F = \{q_1\}$

Transition Table:

δ	a	b
$\rightarrow q_0$	q_1	q_0
$* q_1$	-	q_1

if $(q_0, b) = \frac{ba}{q_1} = q_0$
 $ba, bba, bbb, \dots$

if $(q_1, a) = \frac{aa}{q_0} = q_0$
 $aaa, aab, abb, \dots$

if $(q_1, b) = \frac{ab}{q_1} = q_1$
 $abb, babb, \dots$

what exactly me
 $L = \{w : \frac{w}{a} \in F\}$
 $F = \{q_1\}$

Trap State: A state for which there exists transition itself for all the input symbols chosen from Σ is called a trap state.

In the above DFA, once the machine changes its state to q_2 for any of the input symbols a & b there is no escape from this state. Hence the name trap state.

A trap state can be non final state (q_2).

The non-final trap state is also called dead state.

A final state can also be a trap state but it is not a dead state since the string is accepted at final state.

Ex 5 Obtain DFA to accept strings of a's and b's starting with ab.

Soln: $L = \{ab, aba, abb, abab, \dots\}$

→ ① MS is ab

→ ② $\Sigma = \{a, b\}$

→ ③ Skeleton DFA is



$$\delta(q_0, a) = q_1$$

$$\delta(q_1, b) = q_2$$

→ ④ The other transitions are

$$\delta(q_0, b) = \text{bab} = \text{rejected} = -$$

$$\delta(q_1, a) = \text{aab} = \text{rejected} = -$$

$$\delta(q_2, a) = \text{aba} = \text{accepted} = q_2$$

$$\delta(q_2, b) = \text{abb} = \text{accepted} = q_2$$

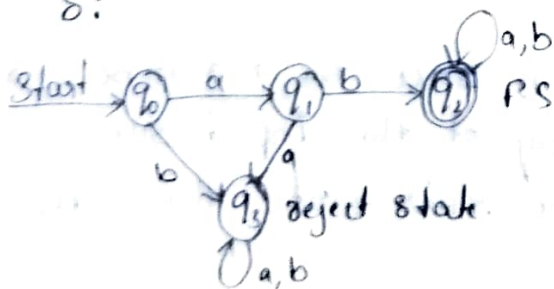
$q_0 = \text{ababab} \dots$
 $q_1 = \text{ababab} \dots$
 $q_2 = \text{ababab} \dots$
 $\text{ababab} \dots$

→ ⑤ $M = (Q, \Sigma, \delta, q_0, F)$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

δ :



$q_0 \rightarrow$ start state

$\{q_2\} \rightarrow F$

Transition table:

δ	a	b
$\rightarrow q_0$	q_1	-
q_1	-	q_2
$* q_2$	q_2	q_2

Ex 6: Draw a DFA to accept strings of a's and b's ending with string abb

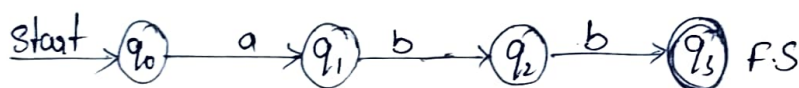
Soln: $L = \{ abb, aabb, babb, \dots \}$

$\rightarrow$ ① MS is abb

$\rightarrow$ ② $\Sigma = \{a, b\}$

$\rightarrow$ ③ Skeleton DFA is

1) $b b b$
 $b b$
 b
 $a a b b x$
 $a a \dots a b b$
 $a b b x$
 $a a x$



$\rightarrow$ $\delta(q_0, a) = q_1$

$\delta(q_1, b) = q_2$

$\delta(q_2, b) = q_3$

3) $a b a a b b$
 $a b a b b$
 $a b a$

$\rightarrow$ ④ The other transitions are

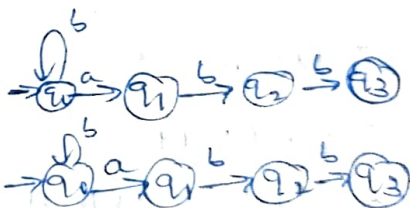
$\delta(q_0, b) = babb = \text{accepted} = q_0$

$\delta(q_1, a) = aabb = \text{accepted} = q_1$

$\delta(q_2, a) = ababb = \text{accepted} = q_1$

$\delta(q_3, a) = abbabb = \text{accepted} = q_1$

$\delta(q_3, b) = abbbabb = \text{accepted} = q_0$



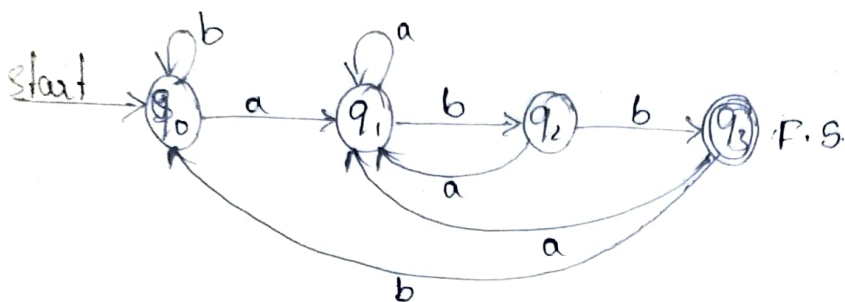
2) q_0
 $b b a b b$
 $a a b b$
 $a a b b$
 $b b a a b b$
 $a a b b$
 $q_1 = b b a b b x$
 $q_3 = b b a a x$

$\rightarrow$ ⑤ $M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{a, b\}$

δ :



$q_0 \rightarrow$ start state

$F = \{q_3\}$

Transition Table:

δ	a	b
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_3
* q_3	q_1	q_0

Ex 7: Draw a DFA to accept strings of 0's and 1's with substrings 01

Soln: $L = \{01, 001, 101, 011, \dots\}$

$\rightarrow$ ① MS is 01

$\rightarrow$ ② $\Sigma = \{0, 1\}$

$\rightarrow$ ③ skeleton DFA is



$$\delta(q_0, 0) = q_1$$

$$\delta(q_1, 1) = q_2$$

$\rightarrow$ ④ The other transitions are

$$\delta(q_0, 1) = q_01 = \text{accepted} = q_0$$

$$\delta(q_1, 0) = 001 = \text{accepted} = q_1$$

$$\delta(q_2, 0) = 010 = \text{accepted} = q_2$$

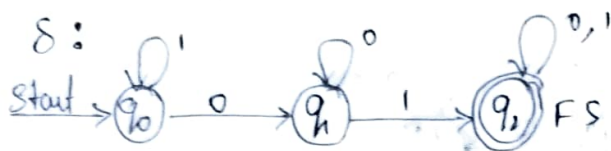
$$\delta(q_2, 1) = 011 = \text{accepted} = q_2$$

→ ⑤ $M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2\}$

$\Sigma = \{0, 1\}$

δ :



$q_0 \rightarrow$ start state

$F = \{q_2\}$

Transition table:

δ	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
$* q_2$	q_2	q_2

ex ⑧: Draw a DFA to accept strings of a's & b's having a sub string aab.

Soln $L = \{a a b, a a b, a a b a, \dots\}$

→ ① MS is aab

→ ② $\Sigma = \{a, b\}$

→ ③ The skeleton DFA is

$L = \{w : w a a b w \text{ where } w \in (a+b)^*\}$



$\delta(q_0, a) = q_1$

$\delta(q_1, a) = q_2$

$\delta(q_2, b) = q_3$

→ ④ The other transitions are

$\delta(q_0, b) = b a a b = \text{accepted} = q_0$

$\delta(q_1, b) = a b a a b = \text{accepted} = q_0$

$\delta(q_2, a) = a a a b = \text{accepted} = q_2$

$\delta(q_3, a) = a a a b b = \text{accepted} = q_3$

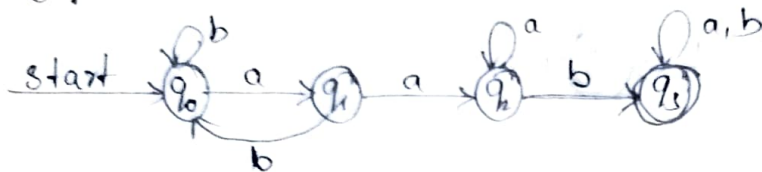
$\delta(q_3, b) = a a b b b = \text{accepted} = q_3$

→ $M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2, q_3\}$

$\Sigma = \{a, b\}$

δ :



$q_0 \rightarrow$ start state

$F = \{q_3\}$

Transition Table:

	δ	a	b
$\rightarrow q_0$	q_1	q_0	
q_1	q_2	q_0	
q_2	q_2	q_3	
* q_3	q_3	q_3	

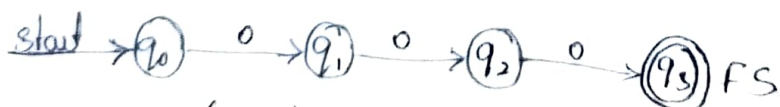
Ex 9: Draw a DFA to accept strings of 0's and 1's having three consecutive 0's.

Soln $L = \{000, 1000, 0001, \underline{1000101}, \dots\}$

→ ① MS is 000

→ ② $\Sigma = \{0, 1\}$

→ ③ The skeleton DFA is



$\delta(q_0, 0) = q_1$

$\delta(q_1, 0) = q_2$

$\delta(q_2, 0) = q_3$

→ ④ The other transitions are

$\delta(q_0, 1) = q_0$

$\delta(q_1, 1) = q_0$

$$\delta(q_2, 1) = q_0$$

$$\delta(q_3, 1) = q_3$$

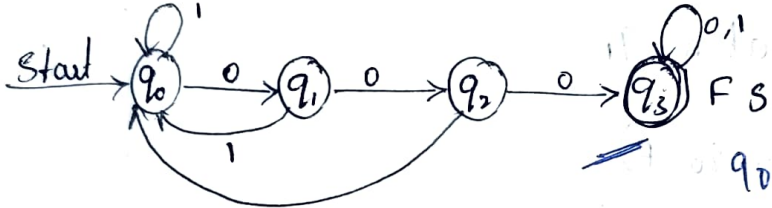
$$\delta(q_3, 0) = q_0$$

$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{0, 1\}$$

δ :



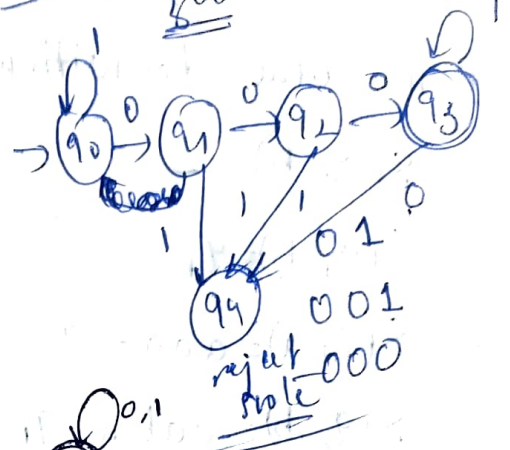
$q_0 \rightarrow$ Start state

$$F = \{q_3\}$$

TT:

δ	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_0
q_2	q_3	q_0
$* q_3$	q_3	q_3

exactly 3 consecutive 1's



$q_0 = 1000$
 $q_1 = 010$

Ex ①: Draw a DFA to accept strings of a's & b's having such that
 $L = \{awa \mid w \in (a+b)^n \text{ where } n \geq 0\}$

Soln: $L = \{aa, aaa, aba, \dots\}$

$\rightarrow$ ① MS is aa

$\rightarrow$ ② $\Sigma = \{a, b\}$

$\rightarrow$ ③ The skeleton DFA is



$$\delta(q_0, a) = q_1$$

$$\delta(q_1, a) = q_2$$

→ ④ The other transitions are

$$\delta(q_0, b) = -$$

$$\delta(q_1, b) = q_1$$

$$\delta(q_2, a) = aaa = q_2$$

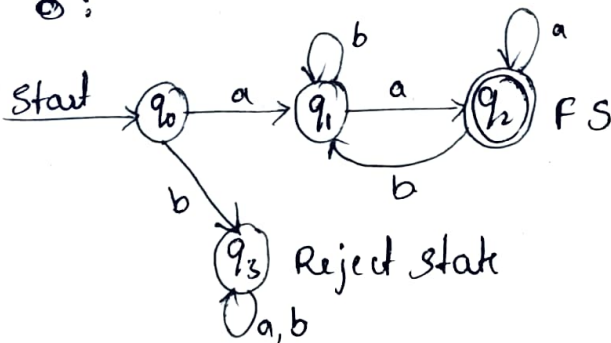
$$\delta(q_2, b) = aab a = q_1$$

→ ⑤ $M = (Q, \Sigma, \delta, q_0, F)$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

δ :



$q_0 \Rightarrow$ start state

$$F = \{q_2\}$$

TT:

δ	a	b
$\rightarrow q_0$	q_1	-
q_1	q_2	q_1
$* q_2$	q_1	q_1

Ex 11: Obtain DFA to accept strings of a's & b's having 4 a's where $\Sigma = \{a, b\}$ (not more than 4 a's) & not consecutive

Sol: $L = \{aaaa, baaaa, aaaaab, \dots\}$

→ ① MG is aaaa

→ ② $\Sigma = \{a, b\}$

→ ③ The skeleton DFA is



$$\delta(q_0, a) = q_1$$

$$\delta(q_1, a) = q_2$$

$$\delta(q_2, a) = q_3$$

$$\delta(q_3, a) = q_4$$

→ ④ The other transitions are

$$\delta(q_0, b) = baaaa = q_0$$

$$\delta(q_1, b) = abaaa = q_1$$

$$\delta(q_2, b) = aabaa = q_2$$

$$\delta(q_3, b) = aaaba = q_3$$

$$\delta(q_4, b) = aaaaab = q_4$$

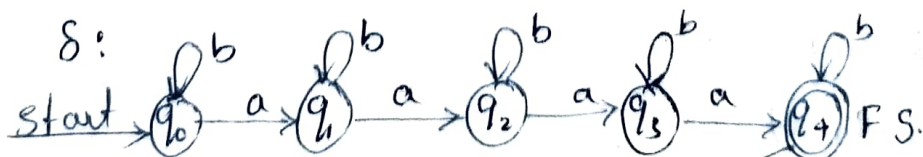
$$\delta(q_4, a) = aaaaaa = -$$

→ ⑤ $M = (Q, \Sigma, \delta, q_0, F)$

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{a, b\}$$

δ :



reject
state

$q_0 \rightarrow$ start state

$$F = \{q_4\}$$

TT:

δ	a	b
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_1
q_2	q_3	q_2
q_3	q_4	q_3
$*q_4$	-	q_4

Ex ②: Obtain DFA to accept strings of 0's, 1's & 2's beginning with no followed by odd number of 1's and ending with 2.

Soln: $L = \{012, 01112, 0111112, \dots\}$

① MS is 012

② $\Sigma = \{0, 1, 2\}$

③ The skeleton DFA is



$$\delta(q_0, 0) = q_1$$

$$\delta(q_1, 1) = q_2$$

$$\delta(q_2, 2) = q_3$$

④ The other transitions are

$$\delta(q_0, 1) = -$$

$$\delta(q_0, 2) = -$$

$$\delta(q_1, 0) = -$$

$$\delta(q_1, 2) = -$$

$$\delta(q_2, 0) = -$$

$$\delta(q_2, 1) = 01112 = q_1$$

Ex ⑬

Soln

①

②

$$\delta(q_3, 0) = -$$

$$\delta(q_3, 1) = -$$

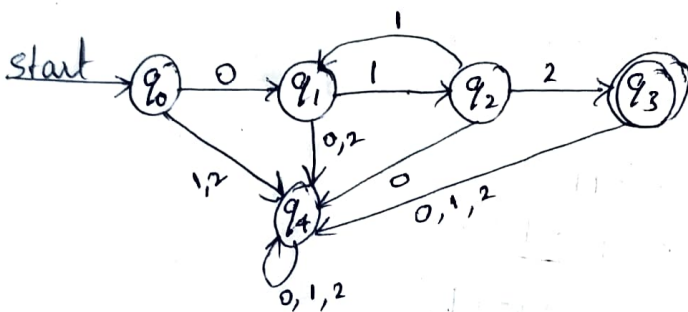
$$\delta(q_3, 2) = -$$

$$\rightarrow \textcircled{5} M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{0, 1, 2\}$$

$$\delta =$$



$q_0 \rightarrow$ start state

$$F = \{q_3\}$$

TT :

	δ	0	1	2
$\rightarrow q_0$		q_1	-	-
q_1		-	q_2	-
q_2		-	q_1	q_3
$* q_3$		-	-	-

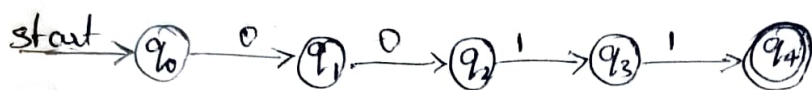
Ex(13): Obtain DFA to accept strings of 0's & 1's starting with atleast 2 0's and ending with atleast 2 1's

Soln $L = \{0011, 0010011, 00101011, \dots\}$

① MS is 0011

② $\Sigma = \{0, 1\}$

③ The skeleton DFA is



$$\delta(q_0, 0) = q_1$$

$$\delta(q_1, 0) = q_2$$

$$\delta(q_2, 1) = q_3$$

$$\delta(q_3, 1) = q_4$$

④ The other transitions are

$$\delta(q_0, 1) = -$$

$$\delta(q_1, 1) = -$$

$$\delta(q_2, 0) = q_2$$

$$\delta(q_3, 0) = q_2$$

$$\delta(q_4, 0) = q_2$$

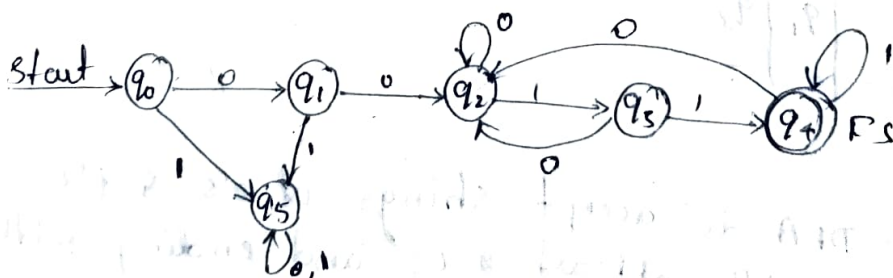
$$\delta(q_4, 1) = q_4$$

⑤ $M = (Q, \Sigma, \delta, q_0, F)$

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{0, 1\}$$

δ :



$q_0 \rightarrow$ start state

$$F = \{q_4\}$$

TT:

δ	0	1
$\rightarrow q_0$	q_1	-
q_1	q_2	-
q_2	q_2	q_3
q_3	q_2	q_4
* q_4	q_2	q_4

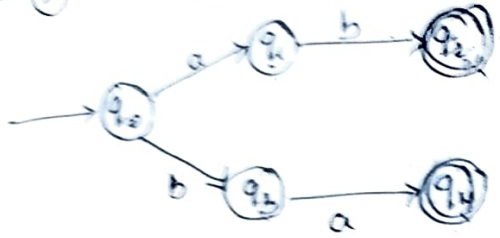
Ques 2: Draw a DFA to accept strings of a's and b's ending with ab or ba

$$L = \{w(ab+ba) \mid w \in (a,b)^*\}$$

1) MS = ab or ba

2) $\Sigma = \{a, b\}$

3) The skeleton DFA is



$$\delta(q_0, a) = q_1$$

$$\delta(q_1, b) = q_2$$

$$\delta(q_2, a) = q_3$$

$$\delta(q_3, a) = q_0$$

4) The other transitions are

$$\delta(q_0, b) = \underline{ab} = q_1$$

$$\delta(q_1, a) = \underline{aba} = q_2$$

$$\delta(q_2, b) = \underline{abb} = q_3$$

$$\delta(q_3, a) = \underline{bba} = q_0$$

$$\delta(q_4, a) = \underline{baa} = q_1$$

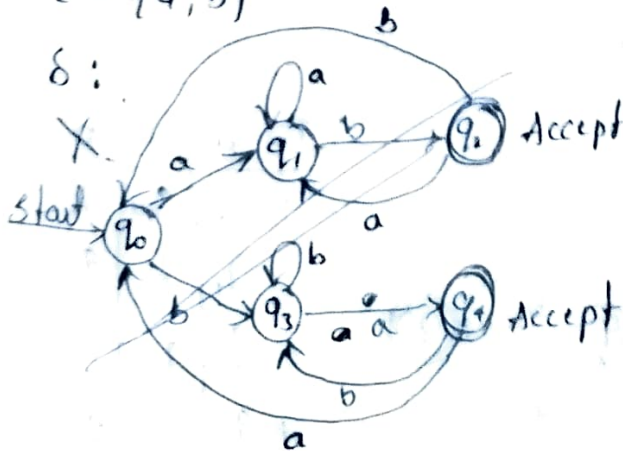
$$\delta(q_4, b) = \underline{bab} = q_2$$

5) $M = (Q, \Sigma, \delta, q_0, F)$

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{a, b\}$$

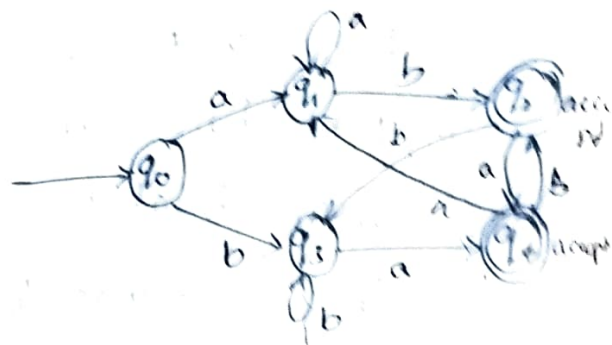
δ :



$q_0 \rightarrow$ start state

$$F = \{q_2, q_4\}$$

abba
abba



TT:

δ	a	b
q_0	q_1	q_3
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_1	q_2
q_4	q_0	q_2

Ex(15): Obtain a DFA to accept the language
 $L = \{wbab \mid w \in \{a,b\}^*\}$

Soln

$$L = \{bab, abab, \dots\}$$

→ (1) MS is bab

→ (2) $\Sigma = \{a, b\}$

→ (3) The skeleton DFA is



$$\delta(q_0, b) = q_1$$

$$\delta(q_1, a) = q_2$$

$$\delta(q_2, b) = q_3$$

→ (4) The other transitions are

$$\delta(q_0, a) = abab = q_0$$

$$\delta(q_1, b) = bbab = q_1$$

$$\delta(q_2, a) = baabab = q_0$$

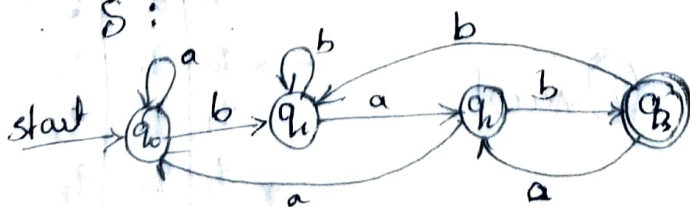
$$\delta(q_3, a) = bababab = q_2$$

$$\delta(q_3, b) = babbab = q_1$$

→ (5) $M = (Q, \Sigma, \delta, q_0, F)$

$$Q = \{q_0, q_1, q_2, q_3\}$$

δ :



$q_0 \rightarrow$ start state

$\{q_3\} \rightarrow F.S.$

<u>TT</u> : δ	a	b
$\rightarrow q_0$	q_0	q_1
q_1	q_2	q_1
q_2	q_0	q_3
$* q_3$	q_2	q_1

Ex 16: Obtain a DFA to accept the strings of 0's & 1's starting with 10 and ends with ^{sub}string 11
 $L = \{1011, 10011, 10111, \dots\}$

① MS is 1011

② $\Sigma = \{0, 1\}$

③ The skeleton DFA is



$$\delta(q_0, 1) = q_1 \quad \delta(q_2, 1) = q_3$$

$$\delta(q_1, 0) = q_2 \quad \delta(q_3, 1) = q_4$$

④ The other transitions are

$$\delta(q_0, 0) = -$$

$$\delta(q_1, 1) = -$$

$$\delta(q_2, 0) = q_2$$

$$\delta(q_3, 0) = 101011 = q_2$$

$$\delta(q_4, 0) = 1011011 = q_2$$

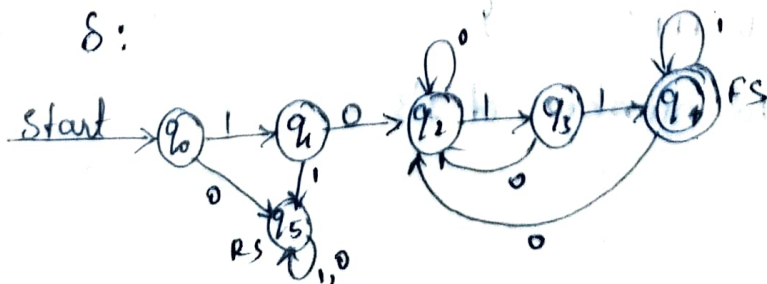
$$\delta(q_4, 1) = 10111 = q_4$$

⑤ $M = (Q, \Sigma, \delta, q_0, F)$

$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{0, 1\}$$

δ :



$q_0 \rightarrow$ Start state

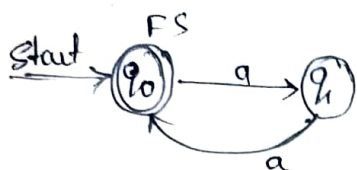
$$F = \{q_4\}$$

TT:

δ	0	1
q_0	-	q_1
q_1	-	q_2
q_2	q_2	q_3
q_3	q_2	q_4
q_4	q_2	q_4

Ex (17): Obtain a DFA to accept the strings of even number of a's over $\Sigma = \{a\}$

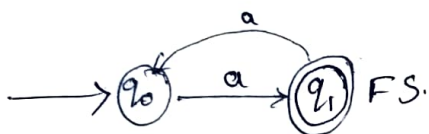
$L = \{aa, aaaa, aaaaaa, \dots\}$



δ	a
* q ₀	q ₁
q ₁	q ₀

Ex (18): Obtain a DFA to accept the strings of odd number of a's over $\Sigma = \{a\}$

$L = \{a, aaa, aaaaa, \dots\}$

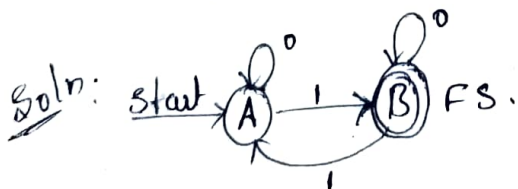


δ	a
q ₀	q ₁
* q ₁	q ₀

Ex (19) Consider the DFA with following transition table describe the language accepted by DFA

Soln:

δ	0	1
$\rightarrow A$	A	B
* B	B	A



$L = \{1, 01, 10, 111, \dots\}$

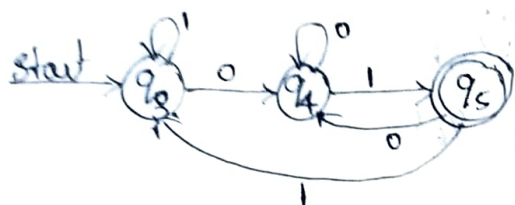
only odd number of 1's

Q8: Draw a DFA to accept set of all strings on the alphabets $\Sigma = \{0,1\}$ that either begin or end or both with substring 01.

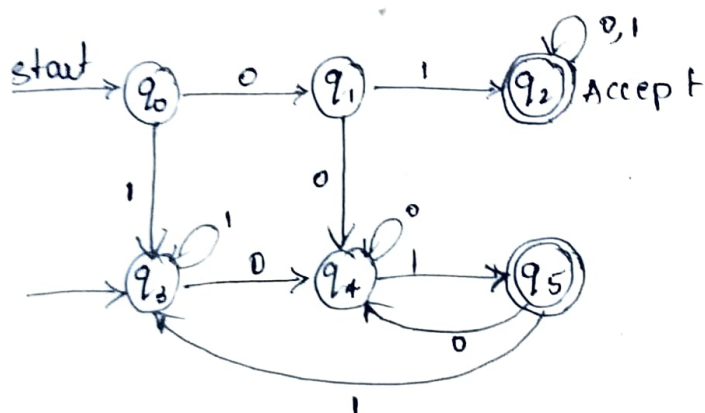
Sol: beginning with 01



Ending with 01



Both :



$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$$

$$\Sigma = \{0, 1\}$$

$$\delta: Q \times \Sigma \rightarrow Q$$

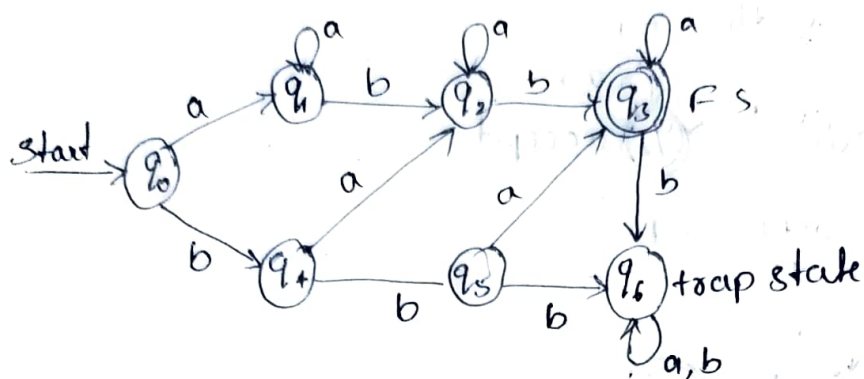
$$F = \{q_2, q_5\}$$

TT:

TT:	δ	0	1
$\rightarrow q_0$	q_1	q_3	
q_1	q_2	q_2	
$\times q_2$	q_2	q_2	
q_3	q_4	q_3	
q_4	q_4	q_5	
$\times q_5$	q_4	q_3	

Ex 21: Draw a DFA to accept $L = \{w : n_a(w) \geq 1, n_b(w) = 2\}$

Soln $L = \{abb, bab, bba, \dots\}$



$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6\}$$

$$\Sigma = \{a, b\}$$

$$SS = q_0$$

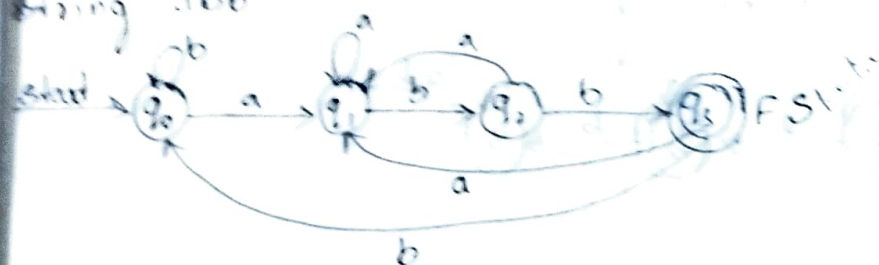
$$FS = \{q_3\}$$

δ	a	b
$\rightarrow q_0$	q_1	q_4
q_1	q_1	q_2
q_2	q_2	q_3
$* q_3$	q_3	-
q_4	q_2	q_5
q_5	q_3	-
q_6	q_6	q_6

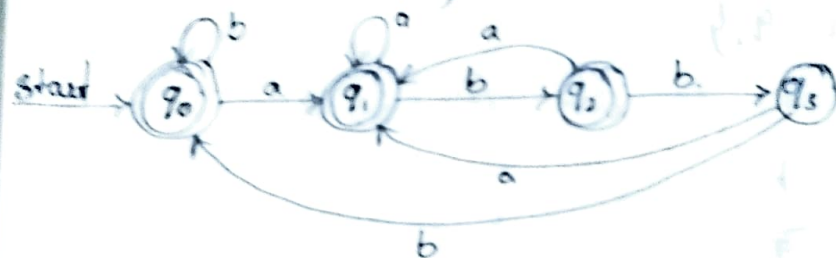
Ex 22: Draw a DFA to accept strings of a's & b's which do not end with the string abb

Soln First design the DFA to accept strings of a's and b's which end with the string abb but make final state to non final state and non final states to final states.

DFA to accept strings of a's & b's end with string abb



DFA to accept strings of a's & b's which do not end with the string abb



$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{a, b\}$$

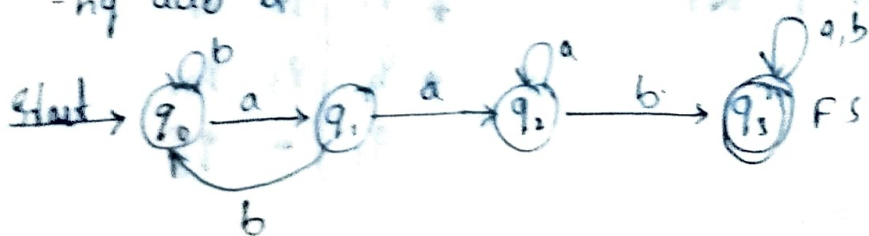
$$FS = \{q_0, q_1, q_2\}$$

TT:

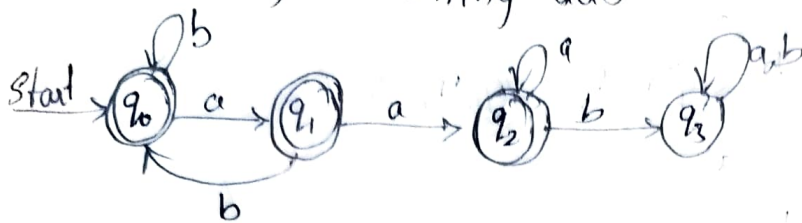
δ	a	b
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_3
q_3	q_1	q_0

ex 23: Draw a DFA to accept strings of a's & b's except those having substring aab

Sol: DFA to accept strings of a's & b's having substring aab is



Now DFA to accept strings of a's & b's except those having substring aab



$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{a, b\}$$

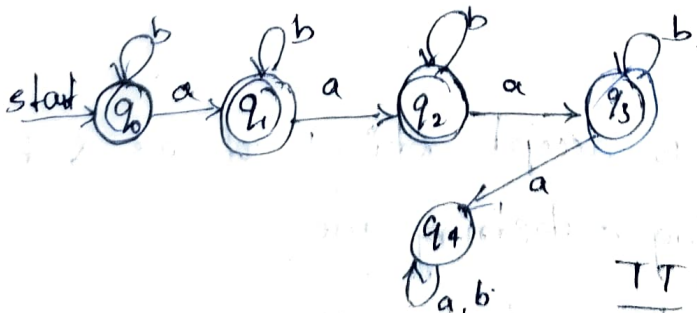
$$F.S = \{q_0, q_1, q_2\}$$

$$S.S = q_0$$

TT:	δ	a	b
$\rightarrow * q_0$		q_1	q_0
$* q_1$		q_2	q_0
$* q_2$		q_3	q_3
q_3		q_3	q_3

Ex 24: Draw a DFA to accept strings of a's & b's having not more than 3 a's ($\leq 3a$)

$$\text{soln: } L = \{\epsilon, aaa, baaa, ababa, a, b, \dots\}$$



$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{a, b\}$$

$$S.S = q_0$$

$$F.S = \{q_0, q_1, q_2\}$$

TT:	δ	a	b
$\rightarrow * q_0$		q_1	q_0
$* q_1$		q_2	q_1
$* q_2$		q_3	q_2
q_3		q_4	q_3
q_4		q_4	q_4

II] Divisible by K Problems:

→ For these type of problems the transitions can be obtained by using the following relation

$$\delta(q_i, a) = q_j \text{ where } j = (r * i + d) \bmod K.$$

- * r is the radix of input for binary $r=2$
- * i is the remainder obtained after dividing ^{by} K
- * d represent digits for binary $d = \{0, 1\}$
- * K is the divisor.

The following are the steps to be followed to find DFA

Step 1: Identify the radix, input alphabets and the divisor.

Step 2: compute the possible remainders, these remainders represent the states of DFA

Step 3: Find the transitions using
 $\delta(q_i, a) = q_j \text{ where } j = (r * i + d) \bmod K$

Step 4: Construct the DFA using the transitions obtained in steps.

✓

Blm

✓

Q10: Construct a DFA which accepts strings of 0's and 1's where the value of each string is represented as a binary number only the string representing zero modulo five should be accepted.

Soln: Steps: $n=2$

$$d = \{0, 1\}$$

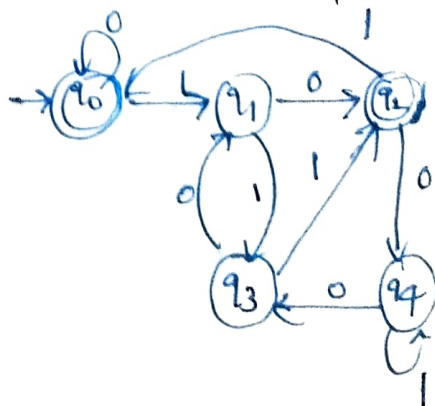
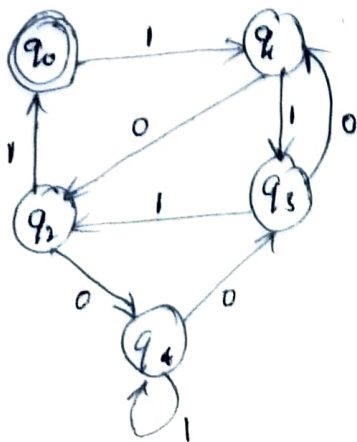
$$k = 5$$

$$\left. \begin{array}{l} 0/5 = 0 \\ 1/5 = 1 \\ 2/5 = 2 \\ 3/5 = 3 \\ 4/5 = 4 \\ 5/5 = 0 \end{array} \right\}$$

Step 2: $i = \{0, 1, 2, 3, 4\}$

$$j = (2*i + d) \bmod k = (2i + d) \bmod k$$

remainder	d	$(2*i + d) \bmod 5 = j$	$\delta(q_i, d) = q_j$
$i = 0$	0	$(2*0 + 0) \bmod 5 = 0$	$\delta(q_0, 0) = q_0$
	1	$(2*0 + 1) \bmod 5 = 1$	$\delta(q_0, 1) = q_1$
$i = 1$	0	$(2*1 + 0) \bmod 5 = 2$	$\delta(q_1, 0) = q_2$
	1	$(2*1 + 1) \bmod 5 = 3$	$\delta(q_1, 1) = q_3$
$i = 2$	0	$(2*2 + 0) \bmod 5 = 4$	$\delta(q_2, 0) = q_4$
	1	$(2*2 + 1) \bmod 5 = 0$	$\delta(q_2, 1) = q_0$
$i = 3$	0	$(2*3 + 0) \bmod 5 = 1$	$\delta(q_3, 0) = q_1$
	1	$(2*3 + 1) \bmod 5 = 2$	$\delta(q_3, 1) = q_2$
$i = 4$	0	$(2*4 + 0) \bmod 5 = 3$	$\delta(q_4, 0) = q_3$
	1	$(2*4 + 1) \bmod 5 = 4$	$\delta(q_4, 1) = q_4$



$$Q = \{q_0, q_1, q_2, q_3, q_4\}$$

$$\Sigma = \{0, 1\}$$

$$q_0 \Rightarrow S.S$$

$$q_0 \Rightarrow F.S.$$

decimal

IV:

δ	0	1
$\rightarrow * q_0$	q_0	q_1
q_1	q_2	q_3
q_2	q_4	q_0
q_3	q_1	q_2
q_4	q_3	q_4

decimal

Ex Q: Draw a DFA to accept strings divisible by 3

Soln:
Step 1: $r = 10$
 $d = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
 $k = 3$

Step 2: $i = \{0, 1, 2\}$
 $j = (10 * i + d) \bmod k = (10i + d) \bmod k$

Step 3: $\{0, 3, 6, 9\}$ with 0 as the remainder.

$\Rightarrow \delta \text{ from } (0, 3, 6, 9) = \delta \text{ from } \{0\}$

$\{1, 4, 7\}$ with 1 as the remainder.

$\Rightarrow \delta \text{ from } (1, 4, 7) = \delta \text{ from } \{1\}$

$\{2, 5, 8\}$ with 2 as the remainder.

$\Rightarrow \delta \text{ from } (2, 5, 8) = \delta \text{ from } \{2\}$

remainders

$i=0$

d

$$(10^i + d) \bmod 3 = j$$

$$\delta(q_i, d) = q_j$$

0

$$(10 * 0 + 0) \bmod 3 = 0$$

$$\delta(q_0, 0) = q_0 \Rightarrow (0, 3, 6, 9)$$

1

$$(10 * 0 + 1) \bmod 3 = 1$$

$$\delta(q_0, 1) = q_1 \Rightarrow (1, 4, 7)$$

2

$$(10 * 0 + 2) \bmod 3 = 2$$

$$\delta(q_0, 2) = q_2 \Rightarrow (2, 5, 8)$$

$i=1$

0

$$(10 * 1 + 0) \bmod 3 = 1$$

$$\delta(q_1, 0) = q_1 \Rightarrow (1, 4, 7)$$

1

$$(10 * 1 + 1) \bmod 3 = 2$$

$$\delta(q_1, 1) = q_2 \Rightarrow (2, 5, 8)$$

2

$$(10 * 1 + 2) \bmod 3 = 0$$

$$\delta(q_1, 2) = q_0 \Rightarrow (0, 3, 6, 9)$$

$i=2$

0

$$(10 * 2 + 0) \bmod 3 = 2$$

$$\delta(q_2, 0) = q_2 \Rightarrow (2, 5, 8)$$

1

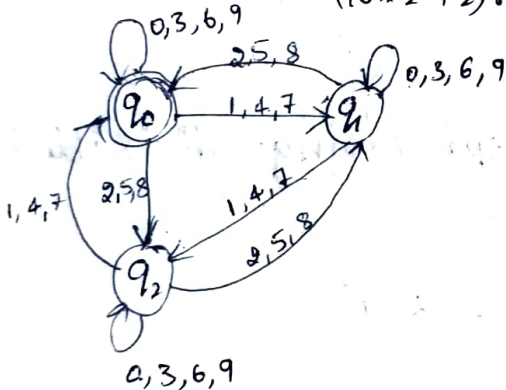
$$(10 * 2 + 1) \bmod 3 = 0$$

$$\delta(q_2, 1) = q_0 \Rightarrow (0, 3, 6, 9)$$

2

$$(10 * 2 + 2) \bmod 3 = 1$$

$$\delta(q_2, 2) = q_1 \Rightarrow (1, 4, 7)$$



$$\frac{128}{3}$$

$$\frac{429}{3}$$

27

$$\frac{18}{3}$$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

$$S \cdot S = q_0$$

$$F \cdot S = q_0$$

A modulo-K counter problem is a type of DFA design problem where the automaton needs to keep track of some count (not of 0's & 1's), length of string etc, modulo K.

that means doesn't need exact remainder count only the

III] Modulo K Counter Problems :

Step 1: Identify the language

Step 2: Identify the number of states

Step 3: Identify start and final state

Step 4: Identify transitions

Step 5: Design DFA by using above transitions

Ex 1: Obtain a DFA to accept strings of even number of a's

Soln:

→ ① $L = \{ w : w \text{ has even no of a's} \}$

→ ② The string w may have even number of a's or odd number of a's and results in following two states

i) strings that accept even no of a's (E)

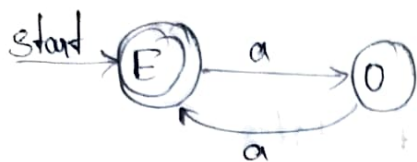
ii) strings that accept odd no of a's (O)

→ ③ Before reading any of the input symbols number of a's will be zero, which represents even number of a's. so the state E with even number of a's is the start state. It is equal to accept even number of a's the state E which accepts even number of a's is the final state

→ ④ $\delta(E, a) = O$

$\delta(O, a) = E$

→ 5



$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{E, O\}$$

$$\Sigma = \{a\}$$

$$\delta(E, a) = O$$

$$\delta(O, a) = E$$

TT :

	δ	a
→ *E	O	
O	E	

Ex 2 Obtain a DFA to accept strings of odd number of a's

Soln:

① $L = \{w \mid w \text{ has odd number of a's}\}$

→ ② case 0: strings that accept odd no of a's (O)

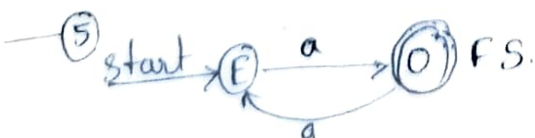
case 1: strings that accept even no of a's (E)

③ Start state: E

Final state: O

$$\delta(E, a) = O$$

$$\delta(O, a) = E$$



$$M = (Q, \Sigma, \delta, E, F)$$

$$Q = \{E, O\}$$

$$\Sigma = \{a\}$$

$$SS = E$$

$$FS = 0$$

TT:

δ	a
$\rightarrow E$	0
*0	E

Ex 3: Obtain DFA to accept the language
 $L = \{w : |w| \bmod 3 = 0\}$ where $\Sigma = \{a\}$ or $L = \{a^{3n} : n \geq 0\}$

Soln: $L = \{w : n_a(w) \text{ are divisible by } 3\}$ where $\Sigma = \{a\}$

Soln: Step 1: The above language indicates that language consists of strings of multiples of 3 a's

Step 2: $|w| \bmod 3$ results in three cases

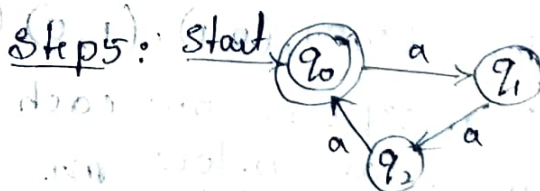
case 1): with remainder 0, the state q_0

case 2): with remainder 1, the state q_1

case 3): with remainder 2, the state q_2

Step 3: Start state: q_0
 Final state: q_0

Step 4: $\delta(q_0, a) = q_1$
 $\delta(q_1, a) = q_2$
 $\delta(q_2, a) = q_0$



$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a\}$$

$$SS = q_0$$

$$FS = q_0$$

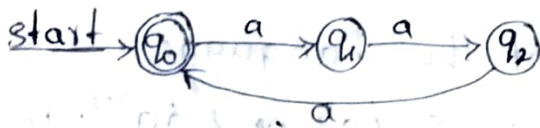
TT:

δ	a
$\rightarrow * q_0$	q_1
q_1	q_2
q_2	q_0

Q: Obtain the DFA to accept the following languages $L = \{w \text{ such that}$
 a) $|w| \bmod 3 \geq |w| \bmod 2$ where $w \in \Sigma^*$ and $\Sigma = \{a\}$
 b) $|w| \bmod 3 \neq |w| \bmod 2$ where $w \in \Sigma^*$ and $\Sigma = \{a\}$

Soln:

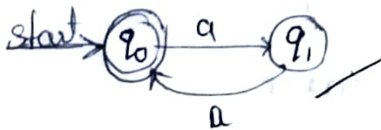
$|w| \bmod 3$:



M_1 :

$\Theta_1 = \{q_0, q_1, q_2\}$

$|w| \bmod 2$



M_2 :

$\Theta_2 = \{q_0, q_1\}$

Transitions of DFA which has strings of w with $|w| \bmod 3$ and $|w| \bmod 2$ can be obtained by taking the cross product of Θ_1 and Θ_2 .

$$\Theta_1 \times \Theta_2 = \{(0,0), (0,1), (1,0), (1,1), (2,0), (2,1)\}$$

Transitions on each of the pair x, y can be obtained as below.

$$\delta((0,0), a) = (\delta(0, a), \delta(0, a)) \Rightarrow q_1 = (1,1)$$

$$\delta((1,1), a) = (\delta(1, a), \delta(1, a)) \Rightarrow q_2 = (2,0)$$

$$\delta((2,0), a) = (\delta(2, a), \delta(0, a)) \Rightarrow q_3 = (0,1)$$

$$\delta((0,1), a) = (\delta(0, a), \delta(1, a)) \Rightarrow q_4 = (1,0)$$

$$\delta((1,0), a) = (\delta(1, a), \delta(0, a)) \Rightarrow q_5 = (2,1)$$

$$\delta((2,1), a) = (\delta(2, a), \delta(1, a)) \Rightarrow q_6 = (0,0)$$

q_6

$$\begin{array}{r} 0 \bmod 3 \quad 0 \bmod 2 \\ 0 \quad 0 \quad 1 \quad 0 \\ \hline 1 \bmod 3 \quad 1 \bmod 2 \\ 1 \quad 1 \quad 1 \\ \hline 2 \bmod 3 \quad 2 \bmod 2 \\ 2 \quad 2 \quad 0 \\ \hline \end{array}$$

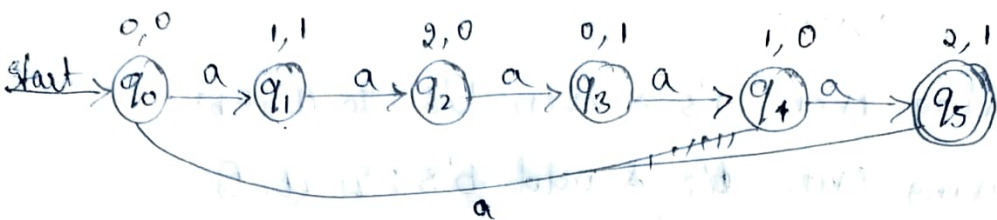
$$\begin{array}{r} 3 \bmod 3 \quad 3 \bmod 2 \\ 0 \quad 1 \quad 1 \quad x \\ 4 \bmod 3 \quad 4 \bmod 2 \\ 1 \quad 0 \quad \checkmark \end{array}$$

Ex 5

Soln

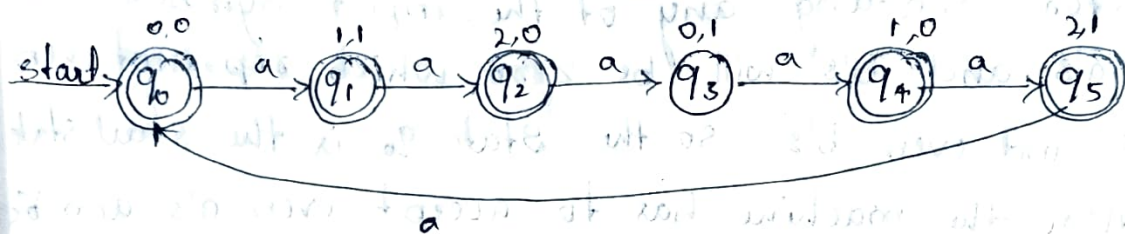
→ 1

→ 2



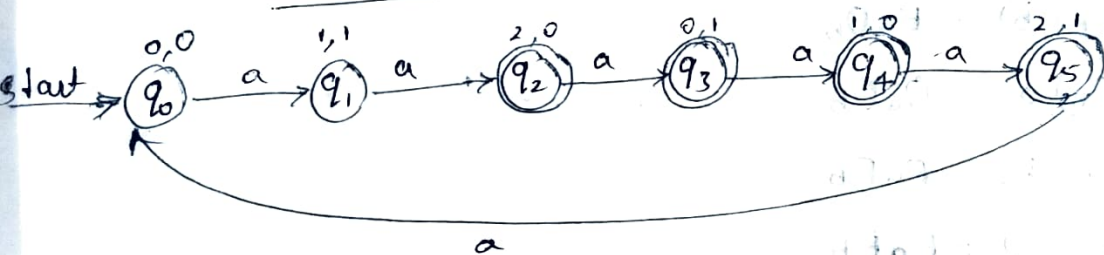
Now i) To accept strings of w such that $|w| \bmod 3 \geq |w| \bmod 2$

$$F = \{(0,0) (1,1) (2,0) (1,0) (2,1)\}$$



ii) To accept strings of w such that $|w| \bmod 3 \neq |w| \bmod 2$

$$F = \{(2,0) (0,1) (1,0) (2,1)\}$$



Ex 5: Obtain a DFA to accept strings of a's and b's having even number of a's and even number of b's

Soln

→ ① $L = \{w : w \text{ have even number of a's and even number of b's}\}$ $L = \{\epsilon, aa, bb, aabb, aabbaa, abab, baab, \dots\}$

→ ② It has 4 possibilities

Even number of a's $\Rightarrow E_a$

Even number of b's $\Rightarrow E_b$

Odd number of a's $\Rightarrow O_a$

Odd number of b's $\Rightarrow O_b$

→ It has 4 cases:

Strings having even a's & even b's: q_0 (EaEb)

Strings having even a's & odd b's: q_1 (EaOb)

Strings having odd a's & even b's: q_2 (OaEb)

Strings having odd a's & odd b's: q_3 (OaOb)

③ Before reading any of the input symbols num of a's and b's will be zero which represent even a's and even b's so the state q_0 is the start state

Here the machine has to accept even a's and b's so that state q_0 with even a's and b's is the final state.

$$\delta((EaEb), a) = OaEb$$

$$\delta((EaEb), b) = EaOb$$

$$\delta((EaOb), a) = OaOb$$

$$\delta((EaOb), b) = EaEb$$

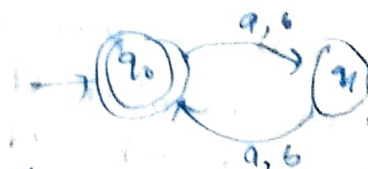
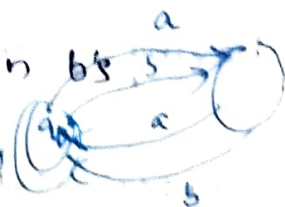
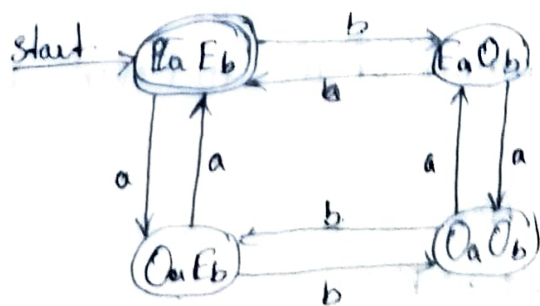
$$\delta((OaEb), a) = EaEb$$

$$\delta((OaEb), b) = OaOb$$

$$\delta((OaOb), a) = EaOb$$

$$\delta((OaOb), b) = OaEb$$

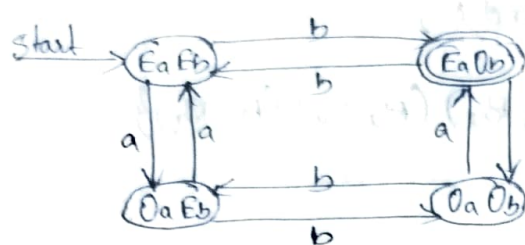
DFA to accept even a's and even b's



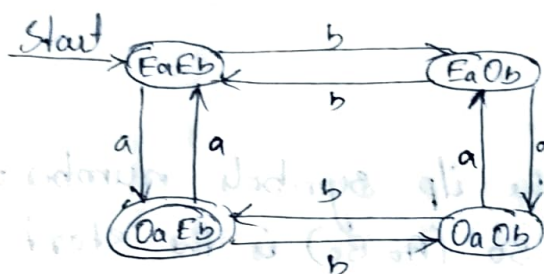
ab
all

aa, ab, ba, bb
a a a a
a b a b
a b a b x

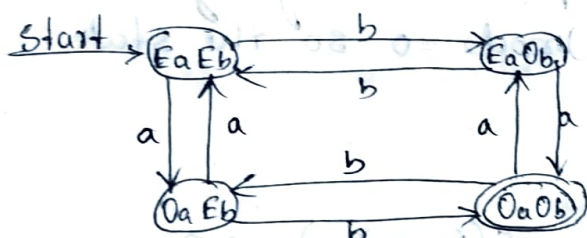
DFA to accept even a's and odd b's



DFA to accept odd a's and even b's



DFA to accept Odd a's and odd b's



3 - 0, 3, 6, 9, 12, 15, 18
2 - 0, 2, 4, 6, 8, 10, 12, 14, 16, 18
 $2 \bmod 2 = 0$
 $2 \bmod 2 = 0$

Ex 6: Obtain a DFA to accept strings of a's & b's such that $L = \{w \mid w \in (a+b)^* \text{ such that}$

$$N_a(w) \bmod 3 = 0$$

$$N_b(w) \bmod 2 = 0 \}$$

Soln:

→ $N_a(w) \bmod 3$ gives the remainder after dividing number of a's by 3

The possible remainders are $\{0, 1, 2\}$ and can be represented as $\Theta_1 = \{A_0, A_1, A_2\}$

→ $N_b(w) \bmod 2$ gives the remainder after dividing number of b's by 2

The possible remainders are $\{0, 1\}$ & can be represented as $\Theta_2 = \{B_0, B_1\}$

→ The possible states of DFA can be obtained by cross product of Θ_1 and Θ_2

$$\Theta_1 \times \Theta_2 = \{(A_0 B_0) (A_0 B_1) (A_1 B_0) (A_1 B_1) (A_2 B_0) (A_2 B_1)\}$$

when $A_0 \Rightarrow N_a(w) \bmod 3 = 0$

$A_1 \Rightarrow N_a(w) \bmod 3 = 1$

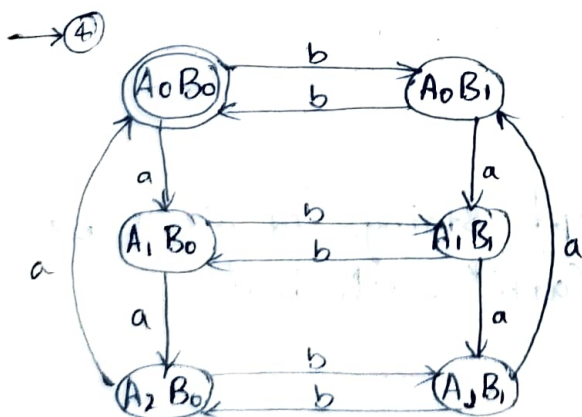
$A_2 \Rightarrow N_a(w) \bmod 3 = 2$

$B_0 \Rightarrow N_b(w) \bmod 2 = 0$

$B_1 \Rightarrow N_b(w) \bmod 2 = 1$

↳ ③ Before reading any of the i/p symbols numbers a's and b's will be zero. So $(A_0 B_0)$ is the start state.

It is required to accept the language $N_a(w) \bmod 3 = 0$ and $N_b(w) \bmod 2 = 0$ so the state $(A_0 B_0)$ is the final state



$$\delta((A_0 B_0), a) = A_1 B_0$$

$$\delta((A_0 B_0), b) = A_0 B_1$$

$$\delta((A_0 B_1), a) = A_1 B_1$$

$$\delta((A_0 B_1), b) = A_0 B_0$$

$$\delta((A_1 B_0), a) = A_2 B_0$$

$$\delta((A_1 B_0), b) = A_1 B_1$$

$$\delta((A_1 B_1), a) = A_2 B_1$$

$$\delta((A_1 B_1), b) = A_1 B_0$$

$$\delta((A_2 B_0), a) = A_0 B_0$$

$$\delta((A_2 B_0), b) = A_2 B_1$$

$$\delta((A_2 B_1), a) = A_0 B_1$$

$$\delta((A_2 B_1), b) = A_2 B_0$$

Applications Of FA:

- ① Design of digital ckt: For designing and checking the behaviour of digital ckt using slw and useful for h/w design such as ckt verification, automatic traffic signals etc.
- ② Compiler construction: Used in the design of lexical analyser which breaks the input text into various units such as identifiers, keywords, punctuation etc.
- ③ String Matching: It is used in designing a slw for identifying the words phrases and other patterns in large bodies of text.
- ④ String processing: To write slw for processing the natural language.
- ⑤ slw Design: In building the slw, to verify the slm having finite number of states.
- ⑥ Other apps: FA are used in variety of applications in artificial intelligence and from knowledge engineering, game theory and computer graphics.

Disadvantage of DFA:

- constructing DFA is difficult
- DFA cannot guess its input
- It is not powerful
- DFA doesn't have the power to be in several states at once.

Advantages of NFA:

- very easy to construct
- It has the ability to guess about its input
- More powerful than DFA

- It has the power to be in several states at once
- It is an efficient mechanism to describe complicated languages.

NFA

Definition: NFA is 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ where

$M \Rightarrow$ name of machine

$Q \Rightarrow$ non empty finite set of states

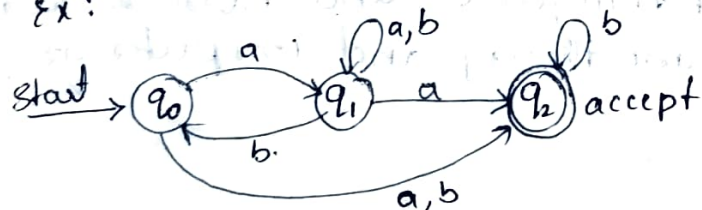
$\Sigma \Rightarrow$ Non empty finite set of input alphabets

$\delta \Rightarrow$ is a function which is mapping from $Q \times \Sigma$ to 2^Q based on the current state and input symbol the machine enters to one or more states.

$q_0 \in Q \Rightarrow S.S$

$F \subseteq Q \Rightarrow F.S.$

Ex:



$Q = \{q_0, q_1, q_2\}$

$2^Q = \{\emptyset, \{q_0\}, \{q_1\}, \{q_2\}, \{q_0, q_1\}, \{q_0, q_2\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}\}$

Transition: change of state after consuming input symbol if there is a change of state from q_i to q_j on input symbol a $\delta(q_i, a) = q_j$

If there is a change of state from q_i to q_j and q_i to q_k on input symbol a $\delta(q_i, a) = \{q_j, q_k\}$

current state	inp	next state	Representation
q_0	a	$\{q_1, q_2\}$	$\delta(q_0, a) = \{q_1, q_2\}$
q_0	b	$\{q_2\}$	$\delta(q_0, b) = q_2$
q_1	a	$\{q_1, q_2\}$	$\delta(q_1, a) = \{q_1, q_2\}$
q_1	b	$\{q_1, q_0\}$	$\delta(q_1, b) = \{q_1, q_0\}$
q_2	a	$\emptyset$	$\delta(q_2, a) = \emptyset$
q_2	b	$\{q_2\}$	$\delta(q_2, b) = q_2$

Extended Transition function: It describes what happens to a state of machine when the input is string. Let $M = (Q, \Sigma, \delta, q_0, F)$ be NFA. The extended transition function is denoted as $\delta^*: Q \times \Sigma^* \rightarrow 2^Q$

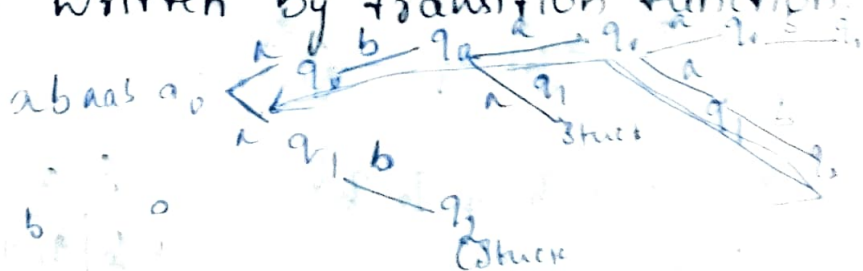
Ex: Change of state from q_i on input string w to set of states $\{r_1, r_2, \dots, r_k\}$ is denoted as $\delta^*(q_i, w) = \{r_1, r_2, \dots, r_k\}$

$\delta^* \Rightarrow$ Extended transition function

$q \Rightarrow$ current state of M .

$w \Rightarrow$ current inp string to be processed

$\{r_1, r_2, \dots, r_k\} \Rightarrow$ the possible states of machine which is written by transition function.



$q_0, q_1, q_2, q_0, q_1, q_2$
 $a \quad b \quad a \quad a \quad b$

Ex ①: NFA to accept strings of a's and b's ending with ab

Soln: $L = \{ab, aab, bab, abab, baab, \dots\}$



$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b\}$$

$$SS = q_0$$

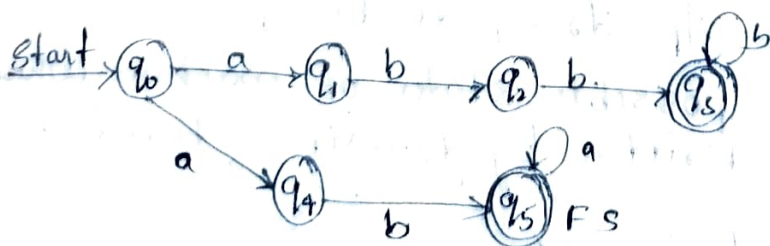
$$FS = q_2$$

δ	a	b
$\rightarrow q_0$	q_0, q_1	q_0
q_1	$\emptyset$	q_2
$* q_2$	$\emptyset$	$\emptyset$

Ex ②: Obtain NFA to accept the following language.

$$L = \{w \mid w \in abab^n \text{ or } ab a^n \text{ where } n \geq 0\}$$

Soln: $L = \{aba, abab, ababb, \dots, ab, aba, abaa, \dots\}$



$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$$

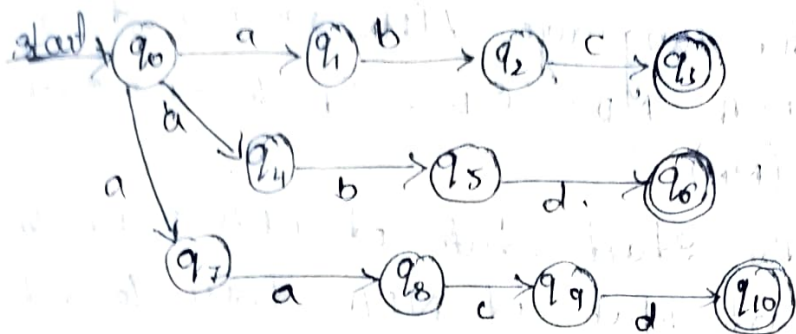
$$\Sigma = \{a, b\}$$

$$SS = q_0$$

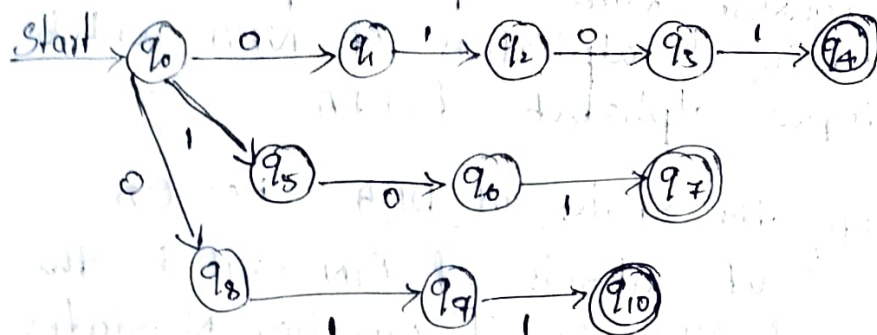
$$F = \{q_3, q_5\}$$

δ	a	b
$\rightarrow q_0$	q_1, q_4	$\emptyset$
q_1	$\emptyset$	q_2
q_2	q_3	$\emptyset$
$* q_3$	$\emptyset$	q_5
q_4	$\emptyset$	q_5
$* q_5$	q_5	$\emptyset$

ex ①: Design an NFA to recognize the following set of strings abc, abd and aacd.



ex ②: Obtain NFA to recognize the following set of strings 0101, 101, 011.



Conversion of NFA to DFA:

- (1) Subset construction
- (2) Lazy evaluation

Subset Construction Method:

Procedure: Given an NFA $M_N = (Q_N, \Sigma, \delta_N, q_0, F_N)$ which accepts the language $L(M_N)$ we can find an equivalent DFA $M_D = (Q_D, \Sigma, \delta_D, q_0, F_D)$ such that $L(M_D) = L(M_N)$.

Step 1: Identify the start state of DFA: If q_0 is the start state of NFA then q_0 is the start state of DFA also.

Step 2: Identify the alphabets of DFA: Since NFA and DFA must accept same language they must consist same input alphabet i.e.; whatever the input alphabets of NFA will be the input alphabets of DFA.

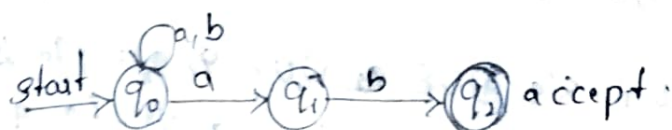
Step 3: Identify the states of DFA: i.e.; Q_D
The set of subsets of Q_N will be the states of DFA so if Q_N has N states then Q_D will have 2^N states.

Step 4: Identify the final states of DFA: i.e.; F_D
if $(q_i, q_j, \dots, q_k)$ is a state in Q_D then $q_i, q_j, \dots, q_k$ will be the final states of DFA if and only if one of the $(q_i, q_j, \dots, q_k)$ is the final state of NFA.

Step 5: Identify the transitions of DFA i.e.; δ_D .
For each state $\{q_i, q_j, \dots, q_k\}$ in Q_D and each input symbol $\{a\}$ is Σ the transition can be obtained as

$$\delta_D(\{q_i, q_j, \dots, q_k\}, a) = \delta_N(q_i, a) \cup \delta_N(q_j, a) \cup \dots \cup \delta_N(q_k, a)$$

ex 0: Obtain the DFA for the following NFA using subset construction method.



Soln: The transition table is

δ	a	b
$\rightarrow q_0$	q_0, q_1	q_0
q_1	$\emptyset$	q_2
$* q_2$	$\emptyset$	$\emptyset$

Step 1: q_0 is the start state of DFA

Step 2: $\Sigma = \{a, b\}$ for DFA

Step 3: $Q_N = \{q_0, q_1, q_2\}$

$$Q_0 = \{\emptyset, q_0, q_1, q_2, \{q_0, q_1\}, \{q_0, q_2\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}\}$$

Step 4: $F_D = \{q_2, \{q_0, q_2\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}\}$

Step 5: For state $\emptyset$ $\delta(\emptyset, a) = \emptyset$
 $\delta(\emptyset, b) = \emptyset$

$$q_0 \Rightarrow \delta(q_0, a) = \{q_0, q_1\}$$

$$\delta(q_0, b) = q_0$$

$$q_1 \Rightarrow \delta(q_1, a) = \emptyset$$

$$\delta(q_1, b) = q_2$$

$$q_2 \Rightarrow \delta(q_2, a) = \emptyset$$

$$\delta(q_2, b) = \emptyset$$

$$\{q_0, q_1\} \Rightarrow \delta(\{q_0, q_1\}, a) = \delta(q_0, a) \cup \delta(q_1, a) = \{q_0, q_1\}$$

$$\delta(\{q_0, q_1\}, b) = \delta(q_0, b) \cup \delta(q_1, b) = \{q_0, q_2\}$$

$$\{q_0, q_2\} \Rightarrow \delta(\{q_0, q_2\}, a) = \delta(q_0, a) \cup \delta(q_2, a) = \{q_0, q_1\}$$

$$\delta(\{q_0, q_2\}, b) = \delta(q_0, b) \cup \delta(q_2, b) = q_0$$

$$\{q_1, q_2\} \Rightarrow \delta(\{q_1, q_2\}, a) = \delta(q_1, a) \cup \delta(q_2, a) = \{ \}$$

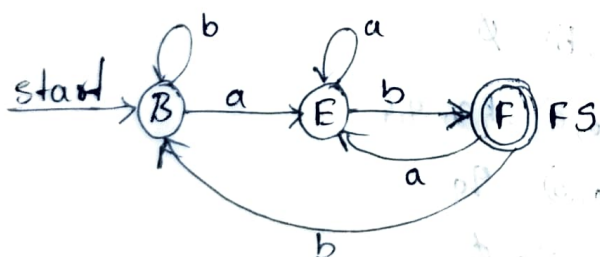
$$\delta(\{q_1, q_2\}, b) = \delta(q_1, b) \cup \delta(q_2, b) = \{q_0, q_1\}$$

$$\{q_0, q_1, q_2\} \Rightarrow \delta(\{q_0, q_1, q_2\}, a) = \delta(q_0, a) \cup \delta(q_1, a) \cup \delta(q_2, a) = \{q_0, q_2\}$$

$$\delta(\{q_0, q_1, q_2\}, b) = \delta(q_0, b) \cup \delta(q_1, b) \cup \delta(q_2, b) = \{q_0, q_2\}$$

δ	a	b
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow q_0$	$\{q_0, q_1\}$	q_0
q_1	$\emptyset$	q_2
* q_2	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
* $\{q_0, q_2\}$	$\{q_0, q_1\}$	q_0
* $\{q_1, q_2\}$	$\emptyset$	q_2
* $\{q_0, q_1, q_2\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$

δ	a	b
A	A	A
$\rightarrow B$	E	B
C	A	D
* D	A	A
E	E	F
* F	E	B
* G	A	D
* H	E	F



Disadvantage of Subset Construction method

→ There are 2^n states and from each state, we have the transition for all symbols in Σ . Hence time complexity to convert NFA to DFA is $\Sigma \times 2^n$.

Since time complexity is exponential the procedure takes very long time to construct this transition. This can be overcome by lazy evaluation method.

2) Easy evaluation method:

Given an NFA $M_N = (Q_N, \Sigma, \delta_N, q_0, F_N)$ which accepts the language $L(M_N)$ we can find an equivalent DFA $M_D = (Q_D, \Sigma, \delta_D, q_0, F_D)$ such that $L(M_D) = L(M_N)$

Step 1: Identify the start state of DFA since q_0 is the start state of NFA, q_0 is the start state of DFA

Step 2: Identify the alphabets of DFA: input alphabets of NFA are the input alphabets of DFA

Step 3: Identify the transitions of DFA for each state $\{q_i, q_j, \dots, q_k\}$ in Q_D and for each input symbol a

$$\delta_D(\{q_i, q_j, \dots, q_k\}, a) = \delta_N(q_i, a) \cup \delta_N(q_j, a) \cup \dots \cup \delta_N(q_k, a) \\ = \{q_l, q_m, \dots, q_n\}$$

Add the state $\{q_l, q_m, \dots, q_n\}$ to Q_D if it is not already in Q_D

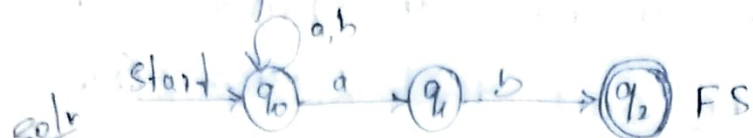
Add the transition from $\{q_i, q_j, \dots, q_k\}$ to $\{q_l, q_m, \dots, q_n\}$ on input symbol a

The steps has to be repeated for each state that is added to Q_D .

Step 4: Identify the final states of DFA if

$\{q_i, q_j, \dots, q_k\}$ if and only if $\{q_i, q_j, \dots, q_k\}$ will be the final state of DFA

Ex ①: Obtain DFA for the following NFA using Lazy evaluation method



① q_0 is the S.S of DFA

② $\Sigma = \{a, b\}$ for DFA

③ $\delta_D(q_0, a) = \{q_0, q_1\}$

$$\delta_D(q_0, b) = q_0$$

$$\rightarrow \delta_D(\{q_0, q_1\}, a) = \delta_D(q_0, a) \cup \delta_D(q_1, a) = \{q_0, q_1\}$$

$$\delta_D(\{q_0, q_1\}, b) = \delta_D(q_0, b) \cup \delta_D(q_1, b) = \{q_0, q_2\}$$

$$\rightarrow \delta_D(\{q_0, q_2\}, a) = \delta_D(q_0, a) \cup \delta_D(q_2, a) = \{q_0, q_1\}$$

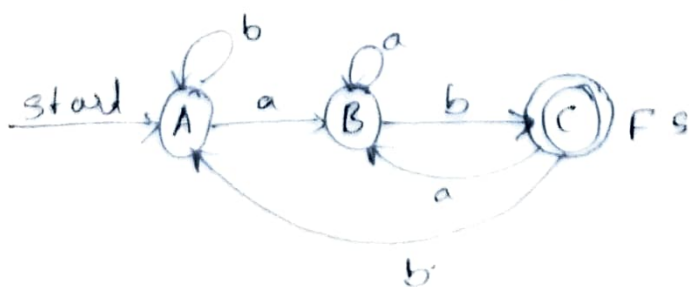
$$\delta_D(\{q_0, q_2\}, b) = \delta_D(q_0, b) \cup \delta_D(q_2, b) = q_0$$

$$\therefore \delta_D = \{q_0, \{q_0, q_1\}, \{q_0, q_2\}\}$$

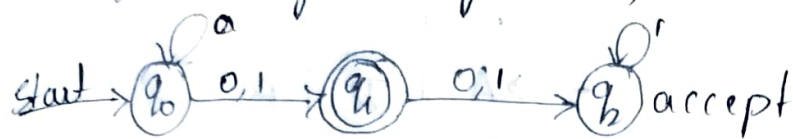
Transition table

δ	δ	a	b
$\rightarrow q_0$	$\{q_0, q_1\}$	q_0	
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_1\}$	
$\{q_0, q_1\}$	$\{q_0, q_1\}$	q_0	

S	a	b
$\rightarrow A$	B	A
B	B	C
*C	B	A



ex ②: Convert the following NFA to its equivalent DFA by using Lazy Evaluation method.



Soln:

δ	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	q_1
* q_1	q_2	q_2
q_2	$\emptyset$	q_2

① q_0 is the SS of DFA

② $\Sigma = \{0, 1\}$

③ Transitions of DFA are

$$\rightarrow \delta(q_0, 0) = \{q_0, q_1\}$$

$$\delta(q_0, 1) = q_1$$

$$\rightarrow \delta(\{q_0, q_1\}, 0) = \delta(q_0, 0) \cup \delta(q_1, 0) = \{q_0, q_1, q_2\}$$

$$\delta(\{q_0, q_1\}, 1) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_1, q_2\}$$

$$\rightarrow \delta(q_1, 0) = q_2$$

$$\delta(q_1, 1) = q_2$$

$$\rightarrow \delta(\{q_0, q_1, q_2\}, 0) = \{q_0, q_1, q_2\}$$

$$\delta(\{q_0, q_1, q_2\}, 1) = \{q_1, q_2\}$$

$$\rightarrow \delta(\{q_1, q_2\}, 0) = q_2$$

$$\delta(\{q_1, q_2\}, 1) = q_2$$

$$\rightarrow \delta(q_2, 0) = \emptyset$$

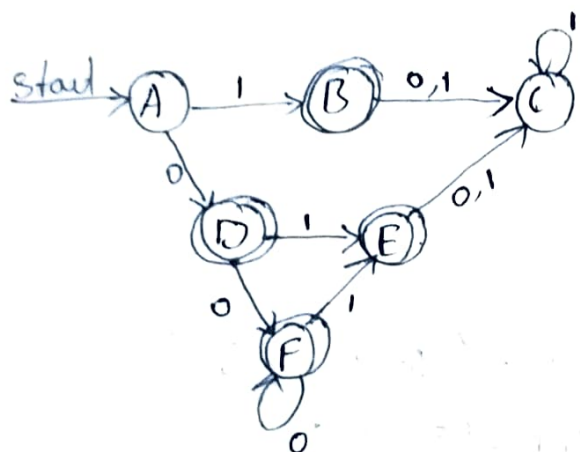
$$\delta(q_2, 1) = q_2$$

$$\therefore \delta_D = \{q_0, q_1, q_2, \{q_0, q_1\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}\}$$

Transition Table:

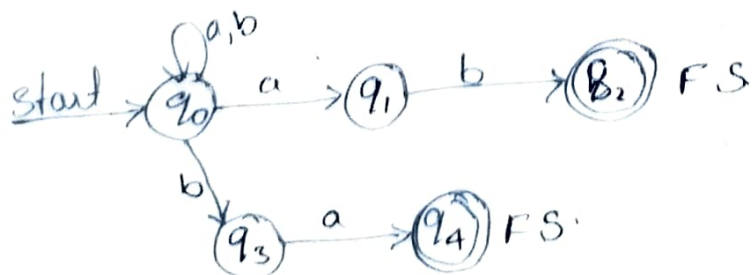
S	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	q_1
* q_1	q_2	q_2
q_2	$\emptyset$	q_2
* $\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$
* $\{q_1, q_2\}$	q_2	q_2
* $\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$

S	0	1
$\rightarrow A$	D	B
* B	C	C
C	$\emptyset$	C
* D	F	E
* E	C	C
* F	F	E



Ex ③: Obtain an NFA to accept strings of a's & b's ending with ab or ba from this NFA
obtain an equivalent DFA

Soln: $L = \{ab, aab, bab, \dots\}$
 $l = \{ba, aba, bba, \dots\}$



δ	a	b
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0, q_3\}$
q_1	ϕ	q_2
$* q_2$	ϕ	ϕ
q_3	q_4	ϕ
$* q_4$	ϕ	ϕ

$\rightarrow \textcircled{1} q_0$ is the SS of DFA

$\rightarrow \textcircled{2} \Sigma = \{a, b\}$

$\rightarrow \textcircled{3}$ The transitions are

$$\rightarrow \delta(q_0, a) = \{q_0, q_1\}$$

$$\delta(q_0, b) = \{q_0, q_3\}$$

$$\rightarrow \delta(\{q_0, q_1\}, a) = \{q_0, q_1\}$$

$$\delta(\{q_0, q_1\}, b) = \{q_0, q_2, q_3\}$$

$$\rightarrow \delta(\{q_0, q_3\}, a) = \{q_0, q_1, q_4\}$$

$$\delta(\{q_0, q_3\}, b) = \{q_0, q_3\}$$

$$\rightarrow \delta(\{q_0, q_2, q_3\}, a) = \{q_0, q_1, q_4\}$$

$$\delta(\{q_0, q_2, q_3\}, b) = \{q_0, q_3\}$$

$$\rightarrow \delta(\{q_0, q_1, q_4\}, a) = \{q_0, q_1\}$$

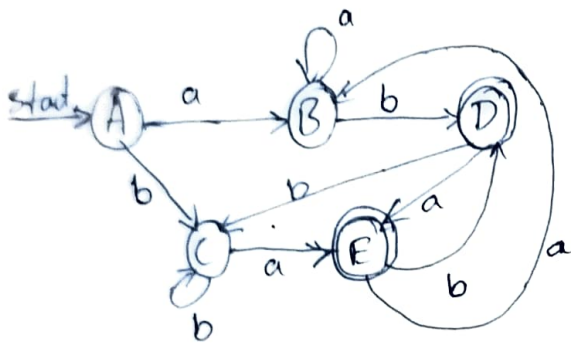
$$\delta(\{q_0, q_1, q_4\}, b) = \{q_0, q_2, q_3\}$$

$$\therefore \Theta_D = \{q_0, \{q_0, q_1\}, \{q_0, q_3\}, \{q_0, q_2, q_3\}, \{q_0, q_1, q_4\}\}$$

transition table:

S	a	b
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0, q_2\}$
$q_0 q_1$	$\{q_0, q_1\}$	$\{q_0, q_2, q_3\}$
$q_0 q_2$	$\{q_0, q_1, q_4\}$	$\{q_0, q_3\}$
* $q_0 q_2 q_3$	$\{q_0, q_1, q_4\}$	$\{q_0, q_3\}$
* $q_0 q_1 q_4$	$\{q_0, q_1\}$	$\{q_0, q_2, q_3\}$

S	a	b
$\rightarrow A$	B	C
B	B	D
C	E	C
* D	E	C
* E	B	D



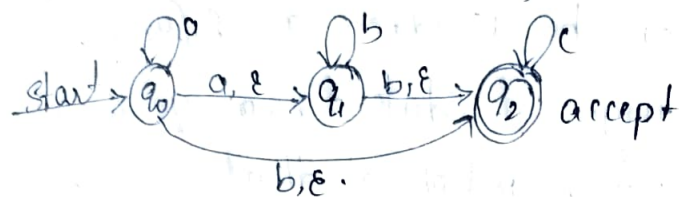
$(a+b)(a+b)$

a

Unit 2: Finite Automaton and Regular Expression

ϵ transitions: Transition with an empty input string is called an epsilon transition

i.e; if there is a transition from one state to another state without any input is called ϵ transition



In the above DFA we have more than 2 transitions from state q_0 with input symbol ϵ so it is an NFA

There is a transition from state q_0 to q_1 with ϵ has the input and another transition from state q_1 to q_2 with ϵ has the input so the above FA is NFA with ϵ transitions.

If zero or more ϵ transitions are present in an NFA then it is an ϵ -NFA.

If there is an ϵ transition then NFA makes transition without receiving any input symbol

ϵ NFA: ϵ NFA is a 5 tuple or quintuple indicating 5 components $M = (Q, \Sigma, \delta, q_0, F)$ where

$Q \Rightarrow$ non empty set of strings

$\Sigma \Rightarrow$ non empty finite set of input symbols

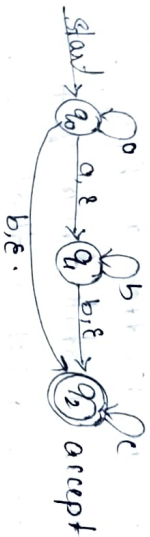
$\delta \Rightarrow$ which is mapping from $Q \times (\Sigma \cup \epsilon)$ to 2^Q based on the current state, there can be a transition to one or more states both input symbols

$q_0 \in Q$ is start state

$F \subseteq Q$ is final state

Unit 2: Finite Automaton and Regular Expression

ϵ transitions: Transition with an empty input string is called an epsilon transition
i.e; if there is a transition from one state to another state without any input is called ϵ transition



In the above DFA we have more than 2 transitions from state q_0 with input symbol ϵ so it is an NFA

There is a transition from state q_0 to q_1 with ϵ has the input and another transition from state q_1 to q_2 with ϵ has the input. So the above FA is NFA with ϵ transitions.

If zero or more ϵ transitions are present in an NFA then it is an ϵ -NFA.

If there is an ϵ transition then NFA makes transition without receiving any input symbol

ϵ NFA: ϵ NFA is a 5 tuple or quintuple indicating 5 components $M = (Q, \Sigma, \delta, q_0, F)$ where

$Q \Rightarrow$ non empty set of strings

$\Sigma \Rightarrow$ non empty finite set of input symbols

$\delta \Rightarrow$ which is mapping from $Q \times (\Sigma \cup \epsilon)^*$ to Q based on the current state, there can be a transition to other state both input symbols

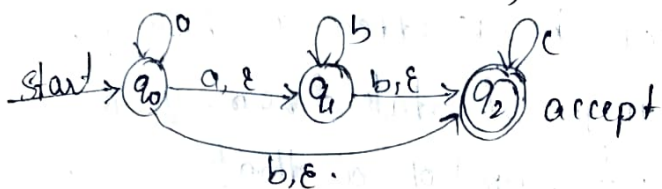
$q_0 \in Q$ is the start state

$F \subseteq Q$ is the final state

Unit 2: Finite Automaton and Regular Expression

ϵ transition: Transition with an empty input string is called an epsilon transition

i.e; if there is a transition from one state to another state without any input is called ϵ transition



In the above DFA we have more than 2 transitions from state q_0 with input symbol ϵ so it is an NFA

There is a transition from state q_0 to q_1 with ϵ has the input and another transition from state q_1 to q_2 with ϵ has the input so the above FA is NFA with ϵ transitions.

If zero or more ϵ transitions are present in an NFA then it is an ϵ -NFA.

If there is an ϵ transition then NFA makes transition without receiving any input symbol

ϵ NFA: ϵ NFA is a 5 tuple or quintuple indicating 5 components $M = (\Theta, \Sigma, \delta, q_0, F)$ where

$\Theta \Rightarrow$ non empty set of strings

$\Sigma \Rightarrow$ non empty finite set of input symbols

$\delta \Rightarrow$ which is mapping from $\Theta \times (\Sigma \cup \epsilon)$ to 2^Θ based on the current state, there can be a transition to other state both input symbols

$q_0 \in \Theta$ is the start state

$F \subseteq \Theta$ is the final state

$$\delta = \{q_0, q_1, q_2\}$$

q_0 = Start state

q_2 = Final state

$$\delta^* = \{\epsilon, \{q_0, q_1\}, \{q_0, q_2\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}\}$$

There is a change of state from q_i to q_j on input symbol 'a' that can be written as $\delta(q_i, a) = q_j$

If there is a change of state from q_i to q_j and q_i to q_k on input symbol 'a' that can be written as $\delta(q_i, a) = \{q_j, q_k\}$

If there is a change of state from q_i to q_j on ϵ written as $\delta(q_i, \epsilon) = q_j$

Current state	input symbol	Next state	Representation	Ex ①
q_0	a	$\{q_0, q_1\}$	$\delta(q_0, a) = \{q_0, q_1\}$	- s
q_0	b	$\{q_2\}$	$\delta(q_0, b) = q_2$	o
q_0	c	$\emptyset$	$\delta(q_0, c) = \emptyset$	
q_0	ϵ	$\{q_0, q_1\}$	$\delta(q_0, \epsilon) = \{q_0, q_1\}$	
q_1	a	$\emptyset$	$\delta(q_1, a) = \emptyset$	Sta
q_1	b	$\{q_1, q_2\}$	$\delta(q_1, b) = \{q_1, q_2\}$	
q_1	c	$\emptyset$	$\delta(q_1, c) = \emptyset$	Tr
q_1	ϵ	q_2	$\delta(q_1, \epsilon) = q_2$	
q_2	a	$\emptyset$	$\delta(q_2, a) = \emptyset$	
q_2	b	$\emptyset$	$\delta(q_2, b) = \emptyset$	
q_2	c	q_2	$\delta(q_2, c) = q_2$	
q_2	ϵ	$\emptyset$	$\delta(q_2, \epsilon) = \emptyset$	

ϵ -closure: ϵ closure of q denoted by $E_{closure}(q)$ is a set of all states which are reachable from q on ϵ transitions only if recursively defined as

i) State q is in $E_{closure}(q)$ i.e. $E_{closure}(q) = q$

if $\text{Eclose}(q)$ contains p and if there is a transition from state p to state r labelled a then r is also in $\text{Eclose}(q)$

$$\text{Eclose}(q_0) = \{q_0, q_1, q_2\}$$

$$\text{Eclose}(q_1) = \{q_1, q_2\}$$

$$\text{Eclose}(q_2) = \{q_2\}$$

Extended transition function of a NFA is denoted by δ^*

The extended transition function describes what happens to a state of the machine when the input is the string $x = (a_1 a_2 \dots a_n)$ in Σ^* .
 NFA let δ is the $\delta^* = \delta(x) = \delta(a_1 a_2 \dots a_n)$

Ex ①: Obtain an ϵ -NFA which accepts string consisting of zero or more a 's followed by one or more b 's followed by one or more c 's.
 $L = \{ \epsilon, a, aa, aaa, \dots, b, bb, bbb, \dots, c, cc, ccc, \dots \}$



Transition table:

δ	a	b	c	ϵ
$\rightarrow q_0$	q_0	$\emptyset$	$\emptyset$	q_1
q_1	$\emptyset$	q_1	$\emptyset$	q_2
$*q_2$	$\emptyset$	$\emptyset$	q_2	$\emptyset$

$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b, c, \epsilon\}$$

$$q_0 \rightarrow \text{start}$$

$$q_2 \rightarrow \text{final state}$$

i) if $Eclose(q)$ contains p and if there is a transition from state ' p ' to state ' r ' labelled ϵ then state r is also an $Eclose(q)$

$$Eclose(q_0) = \{q_0, q_1, q_2\}$$

$$Eclose(q_1) = \{q_1, q_2\}$$

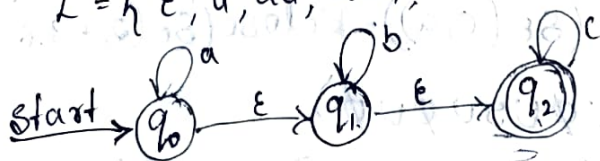
$$Eclose(q_2) = \{q_2\}$$

Extended transition function of ϵ NFA to string

The extended transition function δ^* describes what happens to a state of the machine when the input is the string $M = (\emptyset \leq \delta \leq F)$ be an NFA. Let $\emptyset$ is the $\delta^* = \emptyset x (\Sigma \cup \epsilon)^+$ to $\emptyset$

Ex ①: Obtain an ϵ -NFA which accepts strings consisting of zero or more 'a's followed by zero or more 'b's followed by zero or more 'c's

$$L = \{\epsilon, a, aa, aaa, \dots, b, bb, bbb, \dots, c, cc, \dots, \epsilon\}$$



Transition table:

δ	a	b	c	ϵ
$\rightarrow q_0$	q_0	$\emptyset$	$\emptyset$	q_1
q_1	$\emptyset$	q_1	$\emptyset$	q_2
$*q_2$	$\emptyset$	$\emptyset$	q_2	$\emptyset$

$$M = (\emptyset, \Sigma, \delta, q_0, F)$$

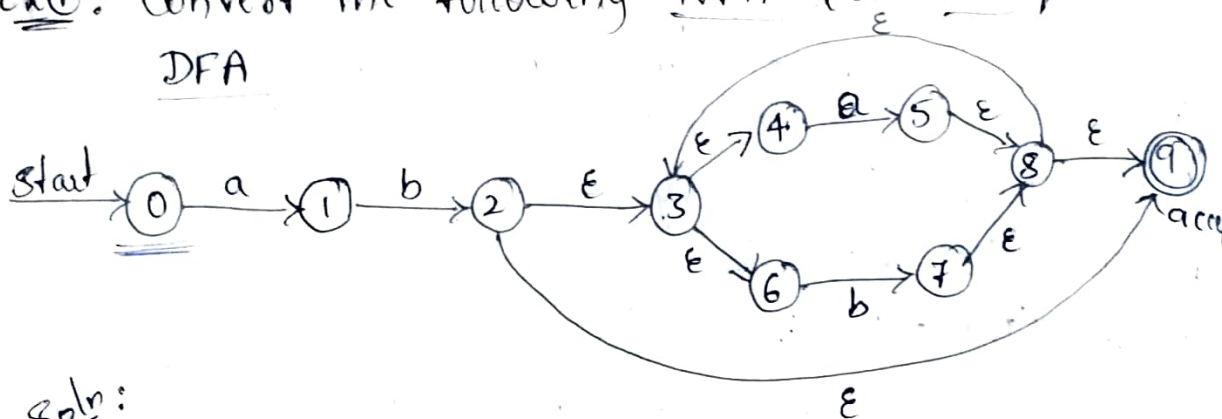
$$\emptyset = \{q_0, q_1, q_2\}$$

$$\Sigma = \{a, b, c, \epsilon\}$$

$$q_0 \Rightarrow S.S$$

q_2 is final state.

Ex 1: Convert the following NFA to its equivalent DFA



Soln:

Step 1: Identify the start state of DFA

$$\text{Eclose}(0) = \{0\} \rightarrow \text{A}$$

Step 2: A

$$A \Rightarrow \delta(A, a) = \text{Eclose}(\delta_\epsilon(A, a)) = \text{Eclose}(1) = \{1\}$$

$$\delta(A, b) = \text{Eclose}(\delta_\epsilon(A, b)) = \text{Eclose}(\emptyset) = \emptyset$$

$$B \Rightarrow \delta(B, a) = \text{Eclose}(\delta_\epsilon(B, a)) = \text{Eclose}(\emptyset) = \emptyset$$

$$\delta(B, b) = \text{Eclose}(\delta_\epsilon(B, b)) = \text{Eclose}(2) = \{2, 3, 4, 6\}$$

$$C \Rightarrow \delta(C, a) = \text{Eclose}(\delta_\epsilon(C, a)) = \text{Eclose}(\delta_\epsilon(2, 4, 3, 6, 9))$$

$$= \text{Eclose}(\delta_\epsilon \emptyset \cup \emptyset \cup 5 \cup \emptyset \cup \emptyset)$$

$$= \text{Eclose}(5) = \{3, 4, 5, 6, 8, 9\} \rightarrow \text{D}$$

$$\delta(C, b) = \text{Eclose}(\delta_\epsilon(C, b)) = \text{Eclose}(\delta_\epsilon(2, 4, 3, 6, 9))$$

$$= \text{Eclose}(\emptyset \cup \emptyset \cup \emptyset \cup 7 \cup \emptyset)$$

$$= \text{Eclose}(7) = \{3, 4, 6, 7, 8, 9\} \rightarrow \text{E}$$

$$D \Rightarrow \delta(D, a) = \text{Eclose}(\delta_\epsilon(D, a)) = \text{Eclose}(\delta_\epsilon(3, 4, 5, 6, 8, 9))$$

$$= \text{Eclose}(\emptyset \cup 5 \cup \emptyset \cup \emptyset \cup \emptyset \cup \emptyset)$$

$$= \text{Eclose}(5) \rightarrow \text{D}$$

$$\delta(D, b) = \text{Eclose}(\delta_\epsilon(D, b)) = \text{Eclose}(\delta_\epsilon(3, 4, 5, 6, 8, 9))$$

$$= \text{Eclose}(\emptyset \cup \emptyset \cup \emptyset \cup 7 \cup \emptyset \cup \emptyset)$$

$$= \text{Eclose}(7) \rightarrow \text{E}$$

$$F \Rightarrow S(F, a) = \text{Eclose}(S(F, a)) = \text{Eclose}(S(F, \{3, 4, 6, 7, 8, 9\}))$$

$$= \text{Eclose}(\emptyset \cup 5 \cup \emptyset \cup \emptyset \cup \emptyset \cup \emptyset)$$

$$= \text{Eclose}(5) \quad \text{--- (D)}$$

$$S(F, b) = \text{Eclose}(S(F, b)) = \text{Eclose}(S(F, \{3, 4, 6, 7, 8, 9\}))$$

$$= \text{Eclose}(\emptyset \cup \emptyset \cup \emptyset \cup 7 \cup \emptyset \cup \emptyset)$$

$$= \text{Eclose}(7) \quad \text{--- (E)}$$

$$\therefore \text{DFA} = \{A, B, C, D, E\}$$

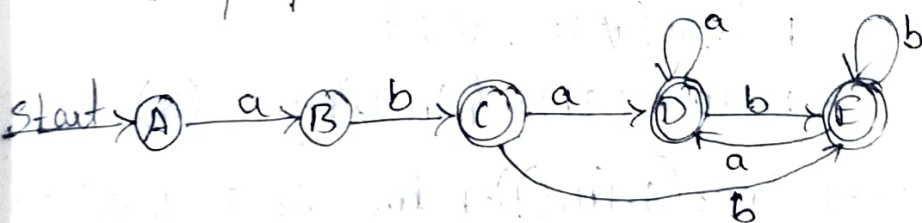
$$\text{DFA} = (\{0\}, \{1\}, \{2, 3, 4, 6, 9\}, \{3, 4, 5, 6, 8, 9\}, \{3, 4, 6, 7, 8, 9\})$$

$$F = \{C, D, E\}$$

Start state: A

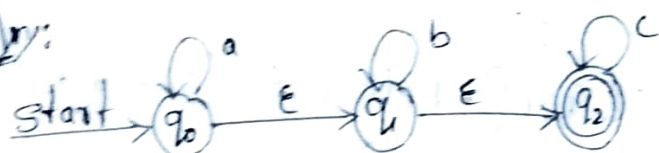
Transition Table:

S	a	b
→ A	B	∅
B	∅	C
* C	D	E
* D	D	E
* E	D	E



Ex ①: Convert the following NFA to its equivalent
-+ DFA

Soln:



Soln:

Step 1: start state of DFA is

$$\text{Eclose}(q_0) = \{q_0, q_1, q_2\} \text{ --- (A)}$$

Step 2:

$$\begin{aligned} A \Rightarrow \delta(A, a) &= \text{Eclose}(\delta_\epsilon(A, a)) = \text{Eclose}(\delta_\epsilon(q_0, q_1, q_2), a) \\ &= \text{Eclose}(\delta_\epsilon(q_0, a) \cup \delta_\epsilon(q_1, a) \cup \delta_\epsilon(q_2, a)) \\ &= \text{Eclose}(q_0 \cup \emptyset \cup \emptyset) \\ &= \text{Eclose}(q_0) = \{q_0, q_1, q_2\} \text{ --- (A)} \end{aligned}$$

$$\begin{aligned} B \Rightarrow \delta(A, b) &= \text{Eclose}(\delta_\epsilon(A, b)) = \text{Eclose}(\delta_\epsilon(q_0, q_1, q_2), b) \\ &= \text{Eclose}(\delta_\epsilon(q_0, b) \cup \delta_\epsilon(q_1, b) \cup \delta_\epsilon(q_2, b)) \\ &= \text{Eclose}(\emptyset \cup \emptyset \cup \emptyset) \\ &= \text{Eclose}(\emptyset) = \emptyset \\ &= \text{Eclose}(q_1) = \{q_1, q_2\} \text{ --- (B)} \end{aligned}$$

$$\begin{aligned} \delta(A, c) &= \text{Eclose}(\delta_\epsilon(A, c)) = \text{Eclose}(\delta_\epsilon(q_0, q_1, q_2), c) \\ &= \text{Eclose}(\delta_\epsilon(q_0, c) \cup \delta_\epsilon(q_1, c) \cup \delta_\epsilon(q_2, c)) \\ &= \text{Eclose}(\emptyset \cup \emptyset \cup q_2) \\ &= \text{Eclose}(q_2) = \{q_2\} \text{ --- (C)} \end{aligned}$$

$$\begin{aligned} B \Rightarrow \delta(B, a) &= \text{Eclose}(\delta_\epsilon(B, a)) = \text{Eclose}(\delta_\epsilon(q_1, q_2), a) \\ &= \text{Eclose}(\delta_\epsilon(q_1, a) \cup \delta_\epsilon(q_2, a)) \\ &= \text{Eclose}(\emptyset \cup \emptyset) \\ &= \emptyset \end{aligned}$$

$$\delta(B, b) = \text{Eclose}(\delta_E(B, b)) = \text{---}(B)$$

$\Rightarrow \delta(\emptyset)$

$$\delta(B, c) = \text{Eclose}(\delta_E(B, c)) = \emptyset$$

$\Rightarrow$ for input a.

$$\delta(c, a) = \text{Eclose}(\delta_E(c, a)) = \text{Eclose}(\delta_E(q_2, a)) = \emptyset$$

$$\delta(c, b) = \text{Eclose}(\delta_E(q_2, b)) = \emptyset$$

$$\delta(c, c) = \text{Eclose}(\delta_E(q_2, c)) = q_2 \text{ ---}(c)$$

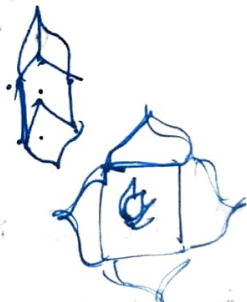
$$\text{DFA} = \{A, B, c\}$$

$$= (\{q_0, q_1, q_2\}, \{q_1, q_2\}, \{q_2\})$$

$$F = \{A, B, c\}$$

$$S = A$$

δ	a	b	c
* A	A	B	c
* B	$\emptyset$	B	c
* c	$\emptyset$	$\emptyset$	c



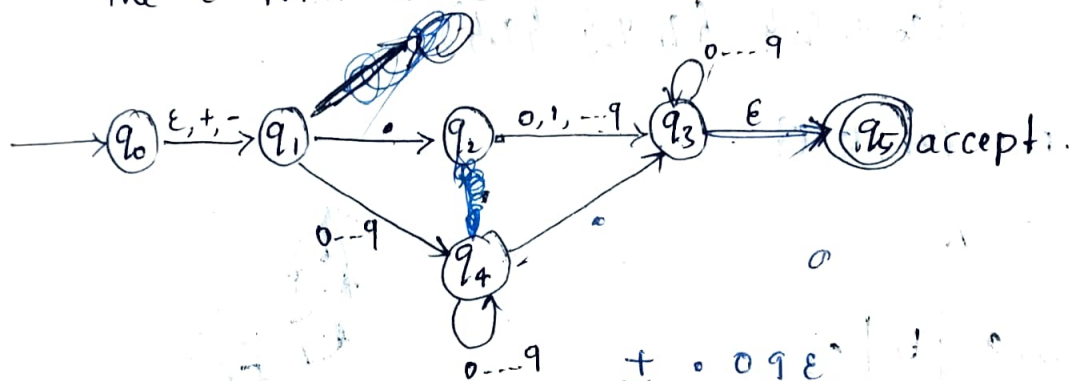
Ex 8 Obtain an NFA with ϵ transitions to accept decimal numbers and obtain $\delta^*(q_0, 4.7)$.

Note: A decimal number has an optional $\pm$ sign followed by $:$ followed string of digits 0-9
 $\rightarrow$ An optional $\pm$ sign followed by a string of digits followed by $:$ and in turn followed by string digits

Soln: $L = \{ \epsilon + - \cdot 0 \dots 9 . \epsilon \}$

$L = \{ \epsilon + - 0 \dots 9 \cdot 0 \dots 9 , \epsilon \}$

The ϵ -NFA is show below



$M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$

$\Sigma = \{ \epsilon, +, -, \cdot, 0 \dots 9 \}$

q_0 is the start state

δ is show below:

δ	ϵ	$+$	$-$	$\cdot$	$0 \dots 9$
q_0	q_1	q_1	q_1	$\emptyset$	$\emptyset$
q_1	$\emptyset$	$\emptyset$	$\emptyset$	q_2	q_4
q_2	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	q_3
q_3	q_5	$\emptyset$	$\emptyset$	$\emptyset$	q_3
q_4	$\emptyset$	$\emptyset$	$\emptyset$	q_3	q_4
q_5	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

$\begin{array}{r} \pm \cdot 09\epsilon \\ \pm 9.99\epsilon \\ \hline \end{array}$

9.
79.79
9

To obtain $\delta^*(q_0, 4.7)$:

$\hookrightarrow$ start state of DFA is q_0

$$\text{Eclose}(q_0) = \{q_0, q_1\}$$

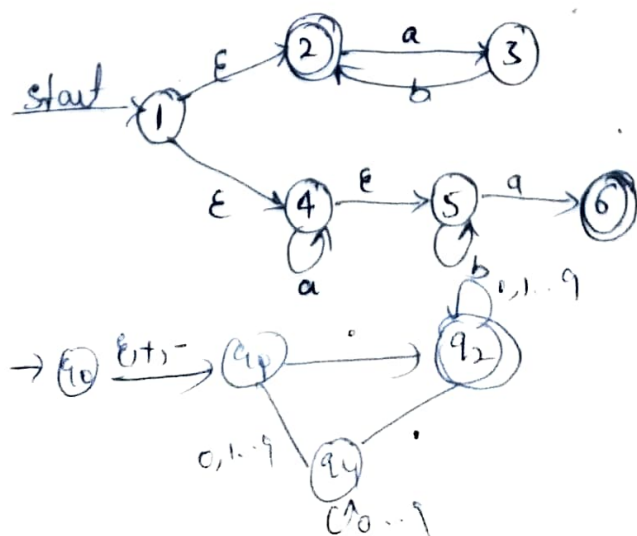
$$\begin{aligned}\delta^*(q_0, 4) &= \text{Eclose}(\delta_\epsilon(\{q_0, q_1\}, 4)) \\ &= \text{Eclose}(\delta_\epsilon(q_0, 4) \cup \delta_\epsilon(q_1, 4)) \\ &= \text{Eclose}(\emptyset \cup \{q_4\}) \\ &= \{q_4\}\end{aligned}$$

$$\begin{aligned}\delta^*(\{q_4\}, \cdot) &= \text{Eclose}(\delta_\epsilon(\{q_4\}, \cdot)) \\ &= \text{Eclose}(\{q_3\}) \\ &= \{q_3, q_5\}\end{aligned}$$

$$\begin{aligned}\delta^*(\{q_3, q_5\}, 7) &= \text{Eclose}(\delta_\epsilon(\{q_3, q_5\}, 7)) \\ &= \text{Eclose}(\{q_3\} \cup \emptyset) \\ &= \text{Eclose}(\{q_3\}) \\ &= \{q_3, q_5\}\end{aligned}$$

$$\boxed{\text{Thus } \delta^*(q_0, 4.7) = \{q_3, q_5\}}$$

ex 4: Convert the following ϵ NFA to DFA



$$\begin{array}{r} - . 012 - \\ + . 012 - \\ + 0.123 - \\ \hline \end{array}$$

Soln

Step 1: The start state of DFA is

$$\text{Eclose}(1) = \{1, 2, 4, 5\} \rightarrow (A)$$

$A \Rightarrow$ for input a

$$\bullet S(A, a) = \text{Eclose}(\delta^*$$

$$\begin{aligned}\delta^*(q_0, 4) &= \text{Eclose}(\delta^E(\{q_0, q_1\}, 4)) \\ &= \text{Eclose}(\delta^E(q_0, 4) \cup \delta^E(q_1, 4)) \\ &= \text{Eclose}(\emptyset \cup q_4) \\ &= \text{Eclose}(q_4)\end{aligned}$$

$$\begin{aligned}\delta^*(q_4, \cdot) &= \text{Eclose}(\delta^E(q_4, \cdot)) \\ &= \text{Eclose}(\{q_3\}) \\ &= \text{Eclose}(q_3)\end{aligned}$$

$$\begin{aligned}\delta^*(q_3, 7) &= \text{Eclose}(\delta^E(q_3, 7)) \\ &= \text{Eclose}(\{q_5\}) \\ &= \{q_5\}\end{aligned}$$

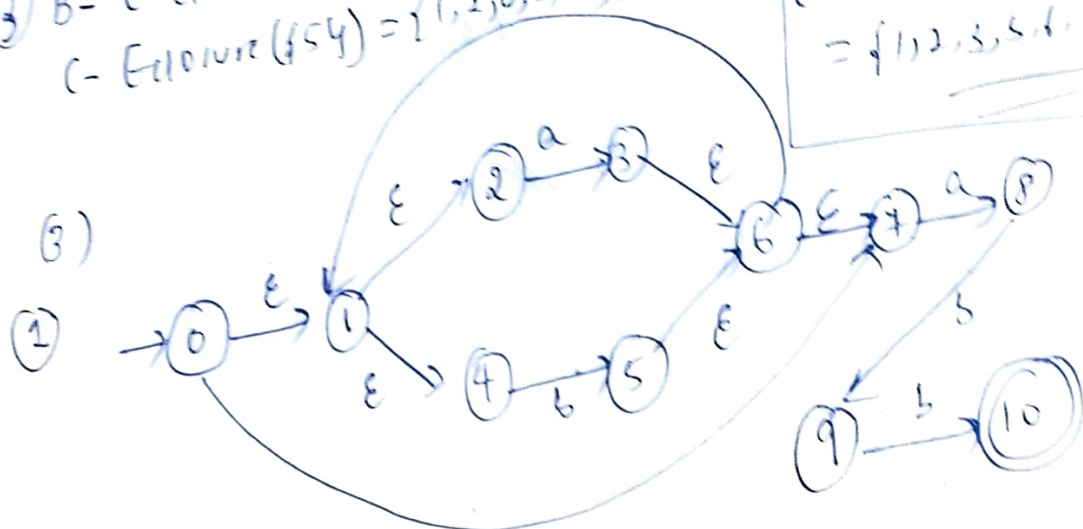
(0111)
1111

way of implementation following by

- ϵ -closure(0) = {0, 1, 2, 4, 7}
 B- ϵ -closure(3, 4) = {1, 2, 3, 4, 6, 7, 8}
 C- ϵ -closure(5, 4) = {1, 2, 4, 5, 6, 7}

ϵ -closure(8, 9, 4) = {1, 2, 3, 4, 6, 7, 8, 9}

ϵ -closure(5, 10) = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}



(A+B)*abb

NFA state	DFA	a		b	
		A	B	C	D
{0, 1, 2, 4, 7}	A	B	C	D	E
{1, 2, 3, 4, 6, 7, 8}	B	B	D	E	F
{1, 2, 4, 5, 6, 7}	C	B	C	E	F
{1, 2, 4, 5, 6, 7, 9}	D	B	E	F	G
{1, 2, 3, 5, 6, 7, 10}	E	B	C	F	G

(1) strings having even length strings & starting with 'a' or odd length strings starting with b.

$$\begin{aligned}
 & a((a+b)(a+b))^* \\
 & b((a+b)(a+b))^* \\
 & ((a+b)(a+b))^*
 \end{aligned}$$

(2) all strings starting with a and ending with b.

$$\begin{aligned}
 & \underline{ab} \quad \underline{aa} \\
 & (a(a+b))^* + (a+b)(a+b)^* \\
 & (ab+aa)(a+b)(a+b)^*
 \end{aligned}$$

aa, bbaa

$L = \{ \epsilon, ab, aa, abaa, abbb, abab, aaaa, \dots \}$

$$\begin{aligned}
 & (ab+aa)(aa+bb+ab+ba)^* \\
 & + \\
 & [(a+b)(a+b)]^* (ab+ba)
 \end{aligned}$$

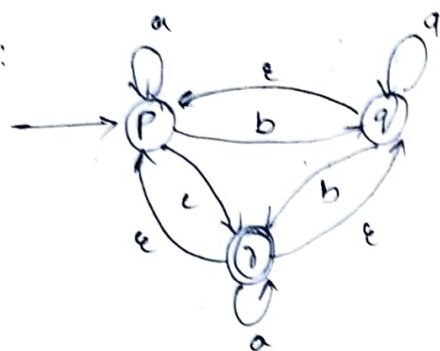
ab

Ques consider the ϵ -NFA

δ	ϵ	a	b	c
$\rightarrow p$	$\emptyset$	p	q	r
q	p	q	r	$\emptyset$
*r	q	r	$\emptyset$	p

- compute ϵ -closure of each state
- Give all the strings of length 3 or less accepted by the automata.
- Convert the ϵ -NFA to DFA

Soln:



i) $E_{\text{close}}(p) = \{p\}$

ii) $E_{\text{close}}(q) = \{q, p\}$

iii) $E_{\text{close}}(r) = \{p, q, r\}$

- ii) The string consist of one c and almost two a's are accepted

$$L = \{c, ac, bcb, ccc, \dots\}$$

When we insert a's to above string at any positions keeping length of 3 or less we find that strings of a's and b's with almost 1 a is accepted.

iii): $Eclose(P) = \{P\} - (A)$

$A \Rightarrow \delta(A, a) = Eclose(P) = \{P\} \quad \text{--- } A$

$\delta(A, b) = Eclose(q) = \{P, q\} \quad \text{--- } B$

$\delta(A, c) = Eclose(\overline{P}, q) = \{P, q, r\} \quad \text{--- } C$

$B \Rightarrow$

$\delta(B, a) = Eclose(P, q) = \{P, q\} \quad \text{--- } B$

$\delta(B, b) = Eclose(P, q, r) = \{q, r, P\} \quad \text{--- } C$

$\delta(B, c) = Eclose(r) = \{P, q, r\} \quad \text{--- } C$

$C \Rightarrow \delta(C, a) = Eclose(P, q, r) = \{P, q, r\} \quad \text{--- } C$

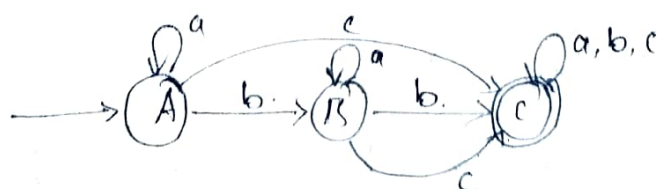
$\delta(C, b) = Eclose(r) = \{P, q, r\} \quad \text{--- } C$

$\delta(C, c) = Eclose(r, P) = \{P, q, r\} \quad \text{--- } C$

$\Phi = \{A, B, C\}$

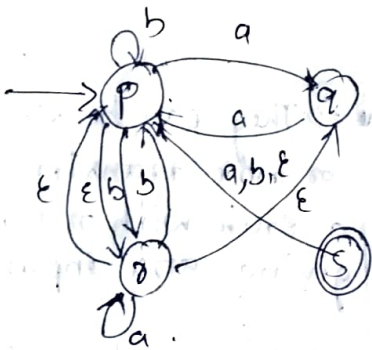
$F = C.$

δ	a	b	c
$\rightarrow A$	A	B	C
B	B	C	C
* C	C	C	C



ex 6) compute ϵ -closure of each state from the following states of NFA

	δ	ϵ	a	b
$\rightarrow$	p	r	q	p, r
	q	$\emptyset$	p	$\emptyset$
	r	p, q	r	p
	*s	p	p	p



Soln $E_{\text{close}}(p) = \{p, q, r\}$

$E_{\text{close}}(q) = \{q\}$

$E_{\text{close}}(r) = \{p, q, r\}$

$E_{\text{close}}(s) = \{p, q, r, s\}$

Differences between DFA, NFA and ϵ -NFA:

DFA	NFA	ϵ -NFA
$\Rightarrow$ DFA is a 5 tuple $M = (Q, \Sigma, \delta, q_0, F)$ where $Q \rightarrow$ Set of States $\Sigma \rightarrow$ Set of input $\delta: Q \times \Sigma \rightarrow Q$ $q_0 \rightarrow$ Start state $F \rightarrow$ Set of final states	$\Rightarrow$ $\delta: Q \times \Sigma \rightarrow 2^Q$	$\Rightarrow$ $\delta: Q \times (\Sigma \cup \epsilon) \rightarrow 2^Q$
$\Rightarrow$ They can be zero or one transition from a state on an input symbol	$\Rightarrow$ They can be zero, one or more transitions from state on an input symbol	$\Rightarrow$ They can be zero, one or more transitions from a state with or without giving any input
$\Rightarrow$ More number of transitions	$\Rightarrow$ Less number of transitions.	$\Rightarrow$ more number of transitions compared to NFA.
$\Rightarrow$ Difficult to construct	$\Rightarrow$ Easy to construct	$\Rightarrow$ Easy to construct using regular expressions
$\Rightarrow$ Less powerful since at any point of time it will be in only one state	$\Rightarrow$ More powerful than DFA since at any point of time it will be in more than one state	$\Rightarrow$ More powerful than NFA since at any point of time it will be in more than one state with or without giving any input

Regular Expressions:

Language accepted by DFA or NFA or ϵ -NFA is called Regular language.

A regular language can be described using Regular expressions consisting of symbols such as alphabets in Σ , operators such as $\cdot, +, \epsilon, *$.

The three operators used to obtain a regular expression are

(+0
(.0
*
Defi
i) ϕ
ii) ϵ
co
iii) a
ou
iv) if
7

The
of th
Exam
Desc
Ostin
of a
Ostin
a a
Ostin
a or
Ostin
of an
null

(+operator) union
(.operator) concatenation

* $\rightarrow$ closure (high p)

Definition: Regular expression

- i) ϕ is a regular expression denoting the empty set.
- ii) ϵ is also a regular expression denoting the empty string.
- iii) a is also a RE that denotes only a .

iv) if R is a RE denoting the language L_R then:

a) $R+S$ is a RE corresponding to the language $L_R \cup L_S$

b) $R.S$ is a regular expression corresponding to the language $L_R.L_S$

c) R^* is a regular expression corresponding to the language L_R^*

The expressions obtained by applying any one of the rules from 1-4 are regular expressions.

Examples of RE and language:

<u>Description</u>	<u>Regular Expression</u>
① String consisting of any number of a 's (0 or more a 's)	a^*
② String consisting of at least one a or	a^+
③ String consisting of either one a or one b	$a+b$
④ Set of strings of a 's and b 's of any length including the null string	$(a+b)^*$

- (+operator) union operator (Least precedence)
- (.operator) concatenation operator (Next least precedence)
- * $\rightarrow$ closure (High precedence)

Definition: Regular expression is recursively defined as

- i) ϕ is regular expression denoting an empty language
- ii) ϵ is also a regular expression that denotes language contains empty string.
- iii) a is also a RE that denotes language contains only a .
- iv) if R is a RE denoting the language L_R & S is a regular expression denoting L_S then:

- a) $R+S$ is a RE corresponding to the language $L_R \cup L_S$
- b) $R \cdot S$ is a regular expression corresponding to the language $L_R \cdot L_S$
- c) R^* is a regular expression corresponding to the language L_R^* .

The expressions obtained by applying any one of the rules from 1-4 are regular expressions.

Examples of RE and language:

<u>Description</u>	<u>Regular Expression</u>
① String consisting of any number of a 's (0 or more a 's)	a^*
② String consisting of at least one a	a^+
③ String consisting of either one a or one b	$a+b$
④ Set of strings of a 's and b 's of any length including the null string	$(a+b)^*$

- ⑥ Set of strings of a's and b's ending with the string abb $(a+b)^*abb$
- ⑦ Set of strings of a's and b's starting with the string ab $ab(a+b)^*$
- ⑧ Set of strings of a's and b's having a substring aa $(a+b)^*aa(a+b)^*$
- ⑨ Set of strings consisting of any number of a's followed by any number of b's followed by any number of c's. $a^*b^*c^*$
- ⑩ Set of strings consisting of at least one a followed by at least one b followed by one c $a^+b^+c^+$
- ⑪ Set of strings of a's and b's ending with a or bb. $(a+b)^*(a+bb)$
- ⑫ Set of strings consisting of even number of a's followed by odd number of b's $(aa)^*(bb)^+b$
- ⑬ Set of strings of 0's and 1's ending with 3 consecutive 0's $(0+1)^*000$
- ⑭ Set of strings consisting of even no of 1's. $(11)^*$
- ⑮ Language represented by string 1 and the string consists of 0's followed by any number of 1's $1+01^*$

15) Language consisting of string 1's
or strings of (01)'s that may
repeat zero or more times

$1+(01)^*$

16) Set of strings consisting of 0
followed by any number of 1's

01^*

Ex 1: Obtain a RE representing strings of a's & b's
having exactly length 2.

$$L = \{aa, ab, ba, bb\}$$

$$RE = (a+b)^2$$

Ex 2: Obtain a RE to obtain strings of a's & b's
of length ≤ 2 .

$$L = \{\epsilon, a, b, aa, ab, ba, bb\}$$

$$RE = (\epsilon + a + b)^2$$

Ex 3: Obtain a RE to ~~to~~ representing strings of
a's and b's of length ≤ 10

$$RE = (\epsilon + a + b)^{10}$$

Ex 4: Obtain a RE of strings of a's and b's having
even length.

$$L = \{aa, aabb, bbaa, \dots\}$$

$$RE = (a+b)(a+b)^*$$

Ex 5: Obtain RE representing strings of a's & b's
having odd length

$$RE = (a+b)(a+b)^*(a+b)$$

Ex 6: Obtain a RE with alternate a's and b's

$L = \{ab, abab, babab, \dots\}$

RE = $(\epsilon + b)(ab)^*(a + \epsilon)$

with $(ab)^*$ we get consecutive a's & b's

Ex 7: Obtain a RE to accept a language consisting of 0's and 1's with at most 1 pair of consecutive zero's

Soln: String may have any number of 1's with no 0's or with one zero or ~~two~~ or with two 0's
(100, 1001, 111100, ...) $L = \{1, 11, 111, \dots, 10, 01, 100, 1100, 0011, 010, 1001, 01010, \dots\}$

→ No zero's which may represent any number of 1's and given by the RE 1^*

→ A zero which is preceded by any combination of 1's and 0's and which is given by the RE $(1+01)^*0(1+01)^*$

→ Two consecutive zero's preceded by any combination of 1's and 0's and followed by combination of 1's and 0's which is given by the RE $(1+01)^*00(1+10)^*$

$\therefore$ RE = $1^* + (1+01)^*0(1+01)^* + (1+01)^*00(1+10)^*$

Ex 8: Obtain a RE to accept a language containing at least 1 a and at least 1 b where $\Sigma = \{a, b\}$

Soln: $L = \{ab, ba, cab, abc, \dots\}$

for ab:

$(a+b+c)^*a(a+b+c)^*b(a+b+c)^*$

for ba:

$$(a+b+c)^* b (a+b+c)^* a (a+b+c)^*$$

$$RE = (a+b+c)^* a (a+b+c)^* b (a+b+c)^* + (a+b+c)^* b (a+b+c)^* a (a+b+c)^*$$

ex 9: Obtain RE such that

$$L(R) = \{w \mid w \in \{0,1\}^* \text{ with at least 3 consecutive 0's}\}$$

Sol: $L = \{000, 1000, 11 \dots 000, 00011, \dots, 0000 \dots, 0 \dots 000\}$

$$RE = (0+1)^* 000 (0+1)^*$$

ex 10: Obtain a RE to accept strings of a's and b's ending with b and end has no substring aa

$$L = \{b, ab, abb, \dots\} \quad L = \{b, ab, abb, b^2ab, \dots\}$$

$$RE = (ab+b)^* (b+ab)$$

ex 11: Obtain a RE to accept strings of 0's and 1's having no two consecutive 0's

$$L = \{01, 0111, \dots\}$$

$$RE = (1+01)^* (0+\epsilon)$$

That is why $(1+01)^* (0+\epsilon)$ is considered.

ex 12: Obtain a RE to accept strings of a's & b's starting with a and ending with b

$$RE = a(a+b)^* b$$

ex 13: Obtain a RE to accept strings of a's & b's whose 2nd symbol from right end is a

$$L = \{aa, ab, bab, aab, \dots\}$$

$$RE = (a+b)^* a (a+b)$$

Ex (14): Obtain RE representing strings of a's and b's whose 10th symbol from right end is a

$$RE = (a+b)^* a (a+b)^9$$

Ex (15): Obtain RE to accept strings of a's and b's such that 3rd symbol from the right is a and 4th symbol from right is b.

$$RE = (a+b)^* ba (a+b)^2$$

Ex (16): Obtain a RE to accept the words or strings with two or more letters but beginning and ending with same letter ab where $\Sigma = \{a, b\}$

$$L = \{aa, bb, abaa, \dots\}$$

$$RE = a(a+b)^* a + b(a+b)^* b$$

Ex (17): Obtain a RE to accept strings of a's and b's whose length is either even or multiple of 3 or both.

Soln

$$RE = ((a+b)(a+b))^* + (a+b)(a+b)(a+b)^*$$

Soln

Ex (18): Obtain a RE to accept strings of a's and b's such that every block of 4 consecutive symbols contains atleast 2 a's

Soln:

i) $aa(a+b)(a+b)$

ii) $a(a+b)a(a+b)$

iii) $a(a+b)(a+b)a$

iv) $(a+b)aa(a+b)$

v) $(a+b)a(a+b)a$

$$vi) (a+b)(a+b)aa$$

$$RE = aa(a+b)(a+b) + a(a+b)a(a+b) + a(a+b)(a+b)a + (a+b)aa(a+b) + (a+b)a(a+b)a + (a+b)(a+b)aa$$

Ex ⑩: Obtain a RE for the language
 $L = \{a^n b^m \mid m+n \text{ is even}\}$

$$RE = (aa)^* (bb)^* + a(aa)^* b(bb)^*$$

Ex ⑪: Obtain RE for the language
 $L = \{a^n b^m \mid m \geq 1, n \geq 1, nm \geq 3\}$

Soln: $nm \geq 3$ if $m=1$ then $n \geq 3 \Rightarrow aaaa^*b$

ii) $nm \geq 3$ if $n=1$ then $m \geq 3 \Rightarrow abbbb^*$

iii) $m \geq 2$ and $n \geq 2 \Rightarrow aaa^*bbb^*$

$$RE = aaaa^*b + abbbb^* + aaa^*bbb^*$$

$aaabbb$
 $aaaaab$

Ex ⑫: Obtain the RE for the language
 $L = \{a^{2n} b^{2m} \mid n \geq 0, m \geq 0\}$

Soln: For every $n \geq 0$ a^{2n} results in even no of a's
 For every $m \geq 0$ b^{2m} results in even no of b's

$(aa)^*$ for even a's

$(bb)^*$ for even b's

$$RE = (aa)^* (bb)^*$$

$m \geq 1, 2 \quad n \geq 1, 2$

$m \geq 1, 2, 3$

$m=1 \quad n=4$

$aaaaab$

$m \quad n$

$n \quad m$

$nm \geq 3$

1 1

1 > 3 X

2 1

2 > 3 X

1 2

2 > 3 X

3 1

3 > 3

1 3

3 > 3

4 1

4 > 3

1 4

4 > 3

Ex 22: Obtain the RE for strings of a's and b's containing not more than 3 a's
 $L = \{ \epsilon, b, aa, aab, baa, abab, baab, bbaa, abba, baab, bbaa \}$
 Solr RE = $b^*(\epsilon + a)b^*(\epsilon + a)b^*(\epsilon + a)b^*$

Ex 23: Obtain the RE for the language
 $L = \{ a^n b^m \mid n \geq 4, m \leq 3 \}$

$$RE = aaaaa^* (b + b^2 + b^3)$$

Ex 24: Obtain a RE for the language

$$L = \{ w : |w| \bmod 3 = 0 \text{ where } w \in (a+b)^* \}$$

$$RE = ((a+b)(a+b)(a+b))^*$$

Ex 25: Obtain a RE for the language

$$L = \{ w : n_a(w) \bmod 3 = 0 \text{ where } w \in \{a, b\}^* \}$$

$$RE = (b^* a b^* a b^* a b^*)^*$$

Ex 26: Obtain a RE for set of all strings that do not end with 01 over $\{0, 1\}^*$

Solr MSE $L = \{ 00, 11, 10, \dots, 0000, 0011, \dots \}$

$$RE = (1+0)^* (00+11+10)$$

Ex 27: Obtain a RE for $L = \{ uvuv : u, v \in \{a, b\}^* \}$

Solr $RE = (a+b)(a+b)(a+b)^*(a+b)(a+b)$

① $RE = (aa+ab+ba+bb)(a+b)^*(aa+ab+ba+bb)$

Ex(8): Give RE for the following languages on $\Sigma = \{a, b, c\}$

- i) all strings containing exactly 1 a
- ii) all strings containing no more than 3 a's
- iii) all strings that contain atleast one occurrence of each symbol in Σ

Soln: 1) $L = \{babcc, bac, a, \dots\}$

$$RE = (b+c)^* a (b+c)^*$$

$$ii) RE = (a+b)^* (\epsilon+a)(a+b)^* (\epsilon+a)(a+b)^* (\epsilon+a)(a+b)^*$$

$$iii) L = \{abc, bca, cba, \dots\}$$

$$RE = abc^+ + bca^+ + cba^+ + bac^+ + tcba^+$$

$$abc^+ = (a+b+c)^* a (a+b+c)^* b (a+b+c)^* c (a+b+c)^*$$

$$bca^+ = (a+b+c)^* b (a+b+c)^* c (a+b+c)^* a (a+b+c)^*$$

$$cab^+ = (a+b+c)^* c (a+b+c)^* a (a+b+c)^* b (a+b+c)^*$$

$$bac^+ = (a+b+c)^* b (a+b+c)^* a (a+b+c)^* c (a+b+c)^*$$

$$acb^+ = (a+b+c)^* a (a+b+c)^* c (a+b+c)^* b (a+b+c)^*$$

$$cba^+ = (a+b+c)^* c (a+b+c)^* b (a+b+c)^* a (a+b+c)^*$$

Ex(29): Obtain the RE for language

$L = \{ \text{strings ends with } ab \text{ or } ba \text{ where } w \in (a,b)^* \}$

Soln: $RE = (a+b)^* \underline{\underline{(ab+ba)}}$

$$a^n b^m$$

$$n \geq 1, m$$

$$m \leq 3$$

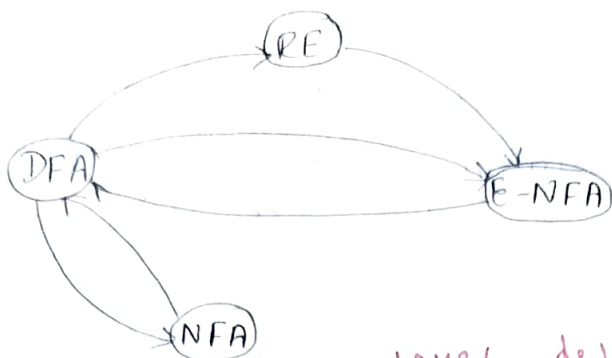
$$\epsilon, b, bb, bbb,$$

$$\underline{\underline{aaaaab}}$$

$$\underline{\underline{aaaaa(a)}}$$

$$(b+bb+bbb+bbbb) (b+bb+bbb+bbbb)$$

Relationship between DFA, NFA, ϵ -NFA and RE:



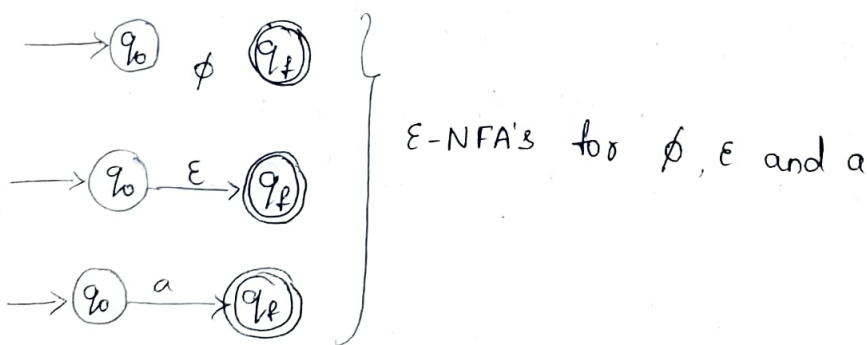
To Obtain ϵ -NFA from RE: *every lang. defined by RE is also defined by a finite automaton.*

Statement: Let R be a RE then there exists a finite automaton $M = (Q \Sigma S q_0 F)$ which accepts $L(R)$

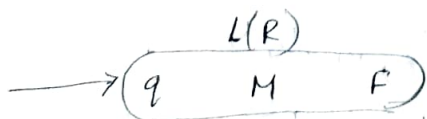
OR

Prove that there exists a finite automaton to accept language $L(R)$ corresponding to the RE R

By definition ϕ, ϵ, a are RE so the corresponding machine to recognize the languages for the respective expressions are shown below:



The Schematic representation of RE are to accept the language $L(R)$ is shown below. where q is start state and f is the FS of the M .



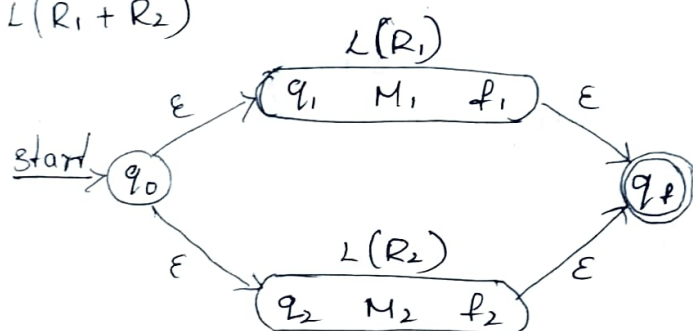
consider two RE R and S then $R+S$, $R.S$ and R^* are RE

Expressions which uses 3 operators + , and *

Let $M_1 = (Q_1, \Sigma, \delta_1, q_1, f_1)$ be a machine which accepts the language $L(R_1)$ corresponding to the RE R_1

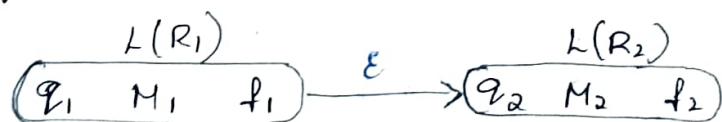
Let $M_2 = (Q_2, \Sigma, \delta_2, q_2, f_2)$ be a machine which accepts the language $L(R_2)$ corresponding to the RE R_2

Case 1: $R = R_1 + R_2$ we can construct an NFA which accepts either $L(R_1)$ or $L(R_2)$ can be represented as $L(R_1 + R_2)$



Here q_0 is the start state of the combined machine and q_f is the final state of the combined machine M .

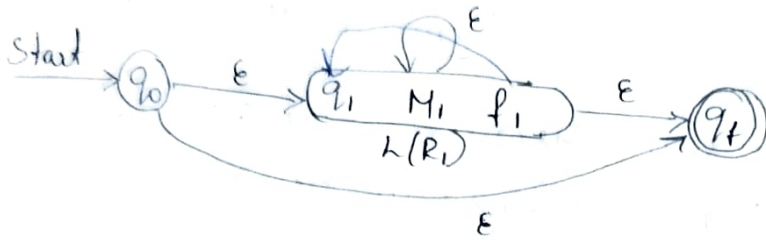
Case 2: $R = R_1 \cdot R_2$; $R = R_1 \cdot R_2$ which accepts $L(R_1)$ followed by $L(R_2)$ which can be represented as $L(R_1 \cdot R_2)$



The machine after accepting $L(R_1)$ moves from q_1 to f_1 . Since there is an ϵ -transition without any input there will be a transition from f_1 to state q_2 in state q_2 upon accepting $L(R_2)$ the machine moves to f_2 which is the final state.

Here q_1 is start state of the combined machine M and f_2 is FS of the combined machine M .

Case 3: $R = (R_1)^*$: Here machine can either accept ϵ or any number of $L(R_1)$. Thus accepting the language $L(R_1)^*$ here q_0 is the start state and q_f is the final state.



Ex 10: Obtain an NFA to accept strings of a's and b's starting with string ab.

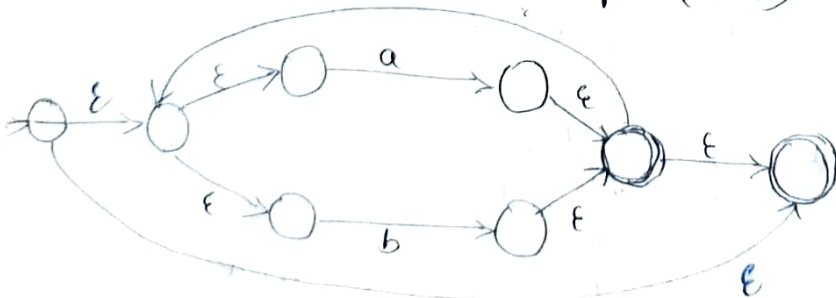
Soln

Step 1: $RE = ab(a+b)^*$

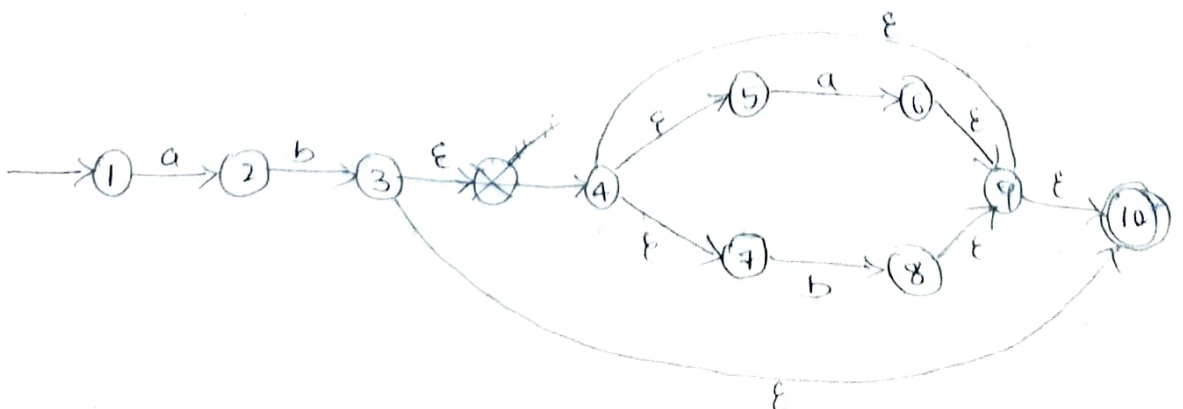
Step 2: Machine to accept ab.



Step 3: Machine to accept $(a+b)^*$



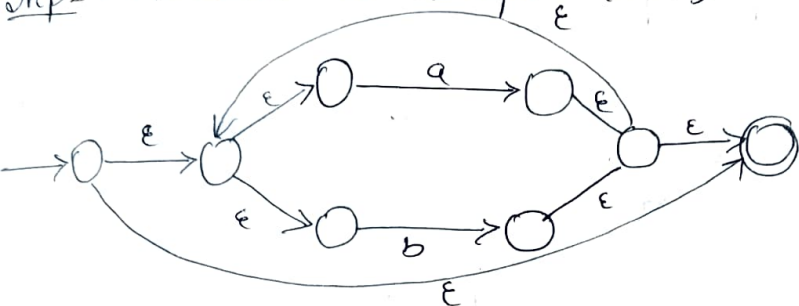
Step 4: Machine to accept $ab(a+b)^*$



Ex ②: Obtain an ϵ -NFA to accept strings of a's and b's ending with string abb

Soln
Step 1: $RE = (a+b)^*abb$

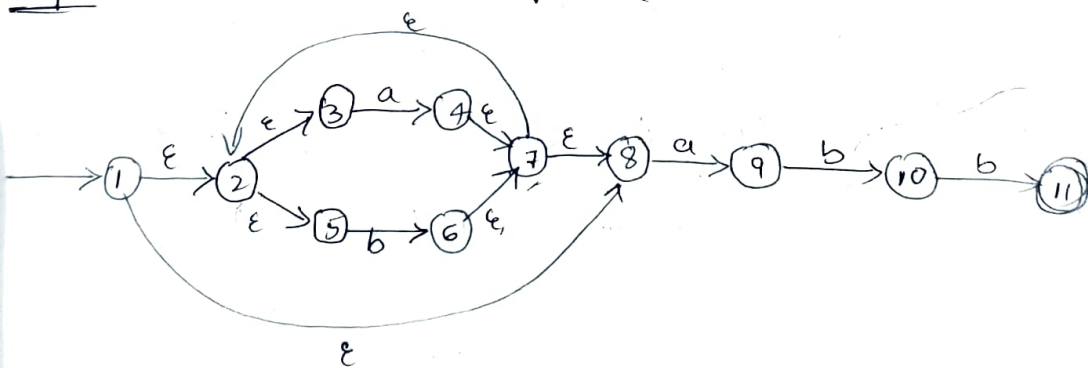
Step 2: Machine to accept $(a+b)^*$



Steps: Machine to accept abb

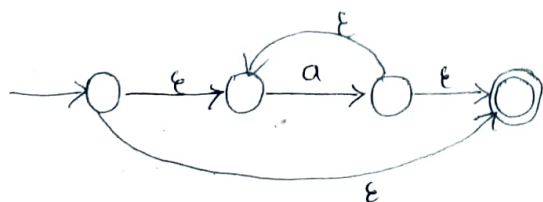


Step 4: Machine to accept $(a+b)^*abb$

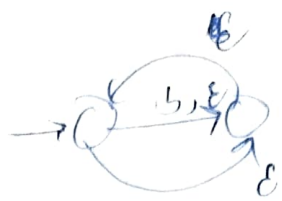
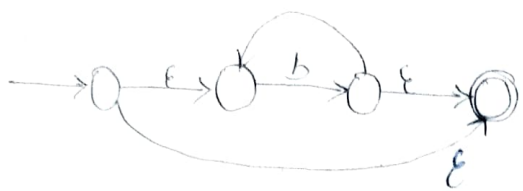


Ex ③: Obtain an NFA for $RE a^* + b^* + c^*$

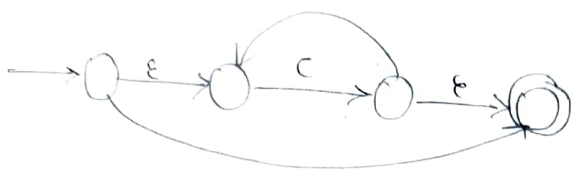
Soln
Step 1: Machine to accept a^*



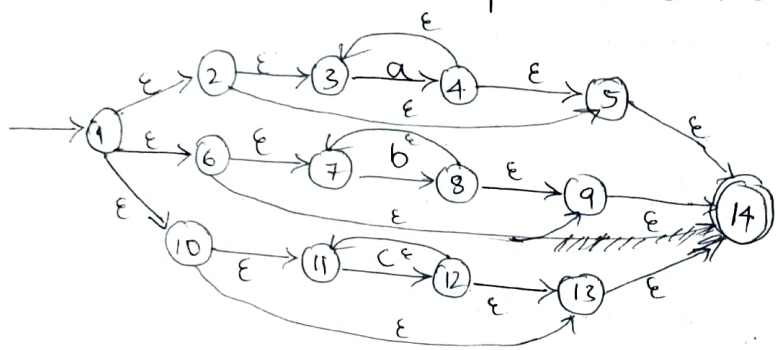
Step 2: Machine to accept b^*



Step 3: Machine to accept c^*

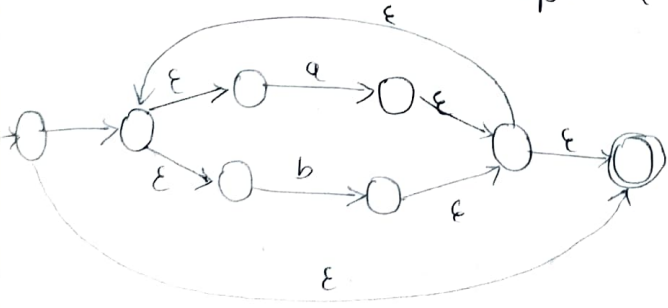


Step 4: Machine to accept $a^* + b^* + c^*$

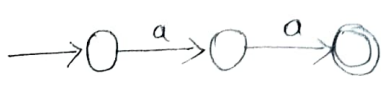


Ex 4 Obtain an NFA for RE $(a+b)^* aa (a+b)^*$

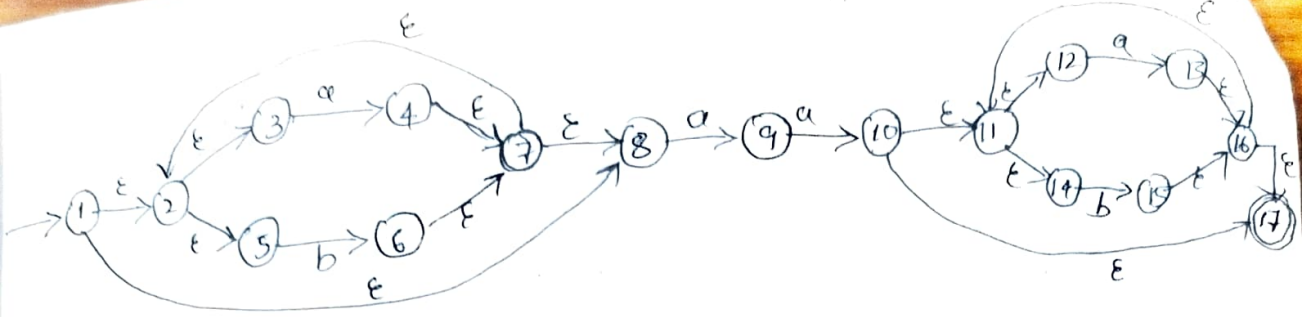
Step 1: Machine to accept $(a+b)^*$



Step 2: Machine to accept aa



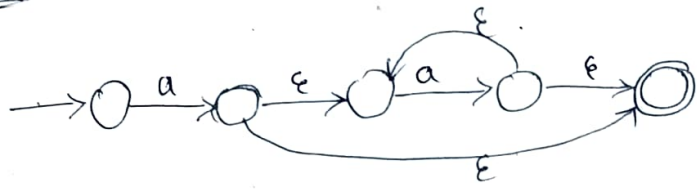
Step 3: Machine to accept $(a+b)^* aa (a+b)^*$



Ex ⑥: Construct an NFA for the RE $aa^* + bb^*$

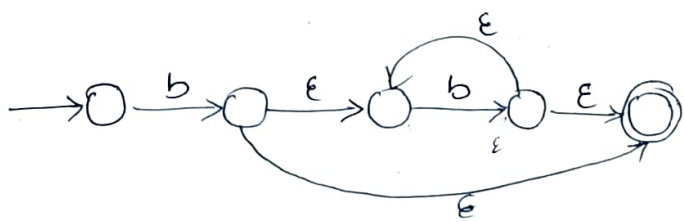
Soln:

Step 1: Machine to accept aa^*

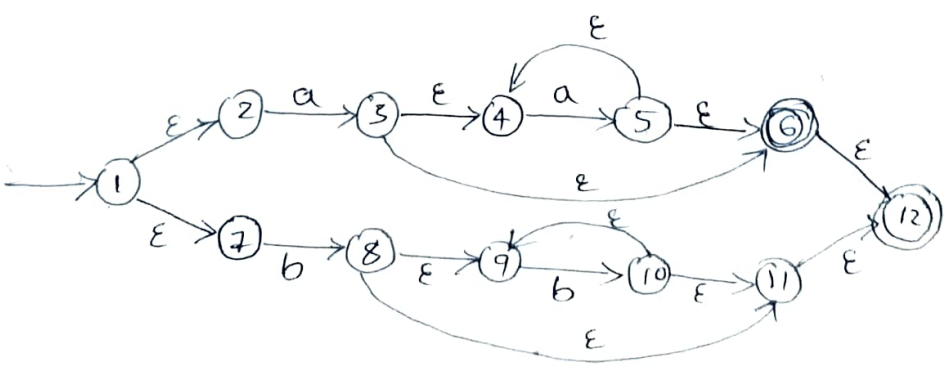


Step 2: Machine to accept bb^*

b, ϵ any no. of

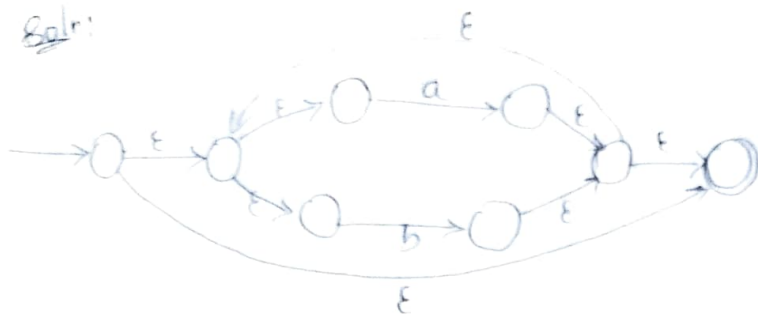


Step 3: Machine to accept $aa^* + bb^*$

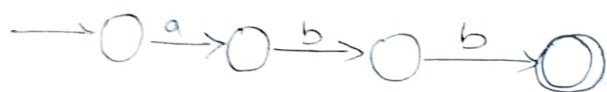


Ex 6): $(a+b)^*abb(a+b)^+$

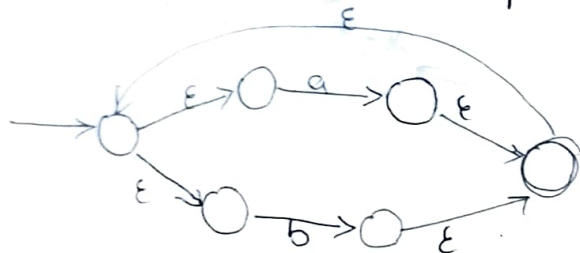
Soln:



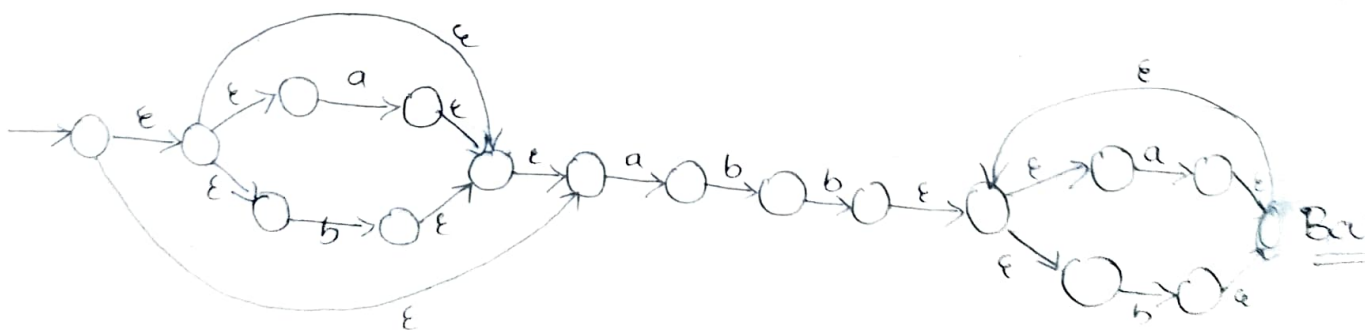
Step 2: Machine to accept abb .



Step 3: Machine to accept $(a+b)^+$



Step 4: Machine to accept $(a+b)^*abb(a+b)^+$



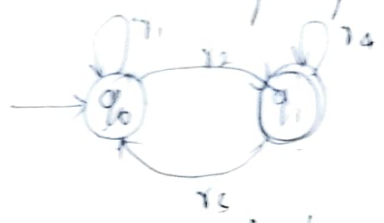
(i) 01+101

(ii) $(01+1)^*$

(iii) 011(0+1)

To obtain RE from FA by state elimination method

Let $M = (Q, \Sigma, \delta, q_0, F)$ be an FA recognizing the language then there exists an equivalent RE for the language L . Such that $L = L(R)$



$$\begin{aligned}
 & x_1^* x_2 + x_1^* x_2 (x_3^* + x_3^* x_4^* x_3^*) \\
 & + x_1^* x_2 x_4^* \\
 & + x_1^* x_2 x_3^* x_2 x_4^* \\
 & + x_1^* x_2 x_3^* x_2
 \end{aligned}$$

generalized transition graph,

where r_1, r_2 and r_3 are the RE and correspond to the labels for the edges. The RE is of the form

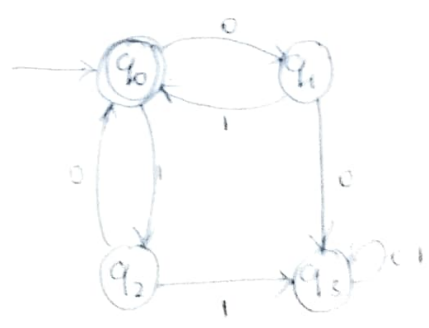
$$R = r_1^* r_2 (r_3^* + r_3^* r_1^* r_2)^* x_1^* x_2 (x_4^* + x_3^* x_1^* x_2 x_4^* + x_3^* x_2)$$

Steps for the state elimination method:

- 1) Identify the deadstate (the state from which there is no way to reach the final state) $x_1, x_2 (x_4^* + x_3^* x_1^* x_2 x_4^* + x_3^* x_2)$
- 2) Remove all the edges from which are connected to dead state and simplify the graphs
- 3) Identify the possible RE from initial to final state.



Ex 1: Obtain the RE for the FA shown below



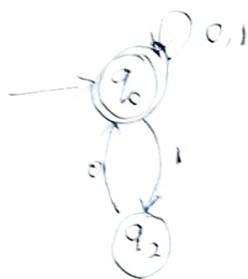
Steps:
 Start state = q_0
 Final state = q_3
 Dead state = q_1

Step 2:



It is clear from the figure that if we put the string 01 the machine goes to state q_1 and comes back to q_0 and the process can be repeated. So instead of q_1 we can consider loop on string (01)

Step 3: $R_1 = (01)^*$



III on 10 the machine comes back to q_0 and the process can be repeated. So instead of q_2 we can consider loop on string 10

Step 4: $R_2 = (10)^*$

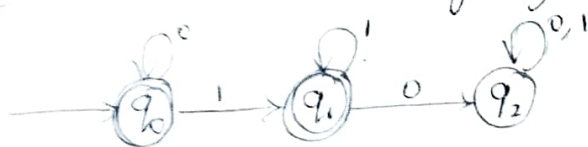


So from above figure it is clear that the machine accepts strings of 01 and 10 of any length and the RE of the form $RE = R_1 + R_2$

$$RE = \underline{(01+10)^*} \text{ or } \underline{(\cancel{01})^* + (\cancel{10})^*}$$

0110, 01, 10,

Ex 2 What is the language accepted by the following



SS $\rightarrow q_0$

FS $\rightarrow q_0, q_1, q_2$

DS $\rightarrow q_2$

Step 2:



Step 3: $R_1 = 0^*$

$R_2 = 0^* \underline{11}^*$

$0^* 1^4$

$RE = R_1 + R_2$

$= 0^* + 0^* 11^*$

$= 0^* (\epsilon + 11^*)$

$= 0^* (\epsilon + 1^+)$

$= \underline{0^* 1^+}$

$= 0^* + 0^* 1^+$

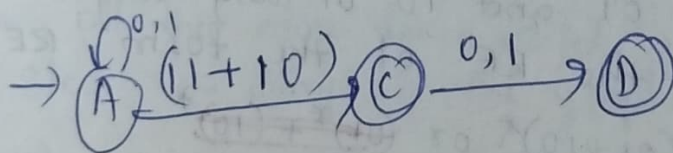
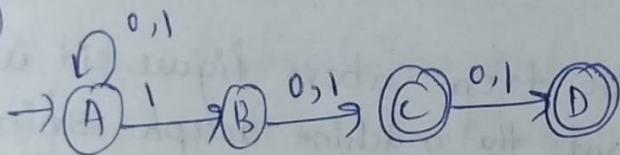
$= 0^* (\epsilon + 1^+)$

$= \underline{\underline{0^* 1^+}}$

Grammar

Set of rules to make the proper sentences

44)



$(0+1)^* [(1+10) + (0+1)]$

$(0+1)^* (1+10) \underline{\underline{[(0+1)]}}$

